BRIEF CONTENTS

CONTENTS IN DETAIL

ACKNOWLEDGMENTS

I've never played in a rock band, but I imagine writing this book is a bit like that. I may have been the singer-songwriter, but this book would have been noticeably inferior had it not been for the exceptional talents and support of the following people.

Jeremy Bowers is one of the most talented engineers and enjoyable human beings I've had the pleasure of knowing. The depth and breadth of his knowledge considerably eased my impostor syndrome, knowing that he staked his reputation and likely his career on the success of this book. He reviewed every paragraph and line of code to ensure their coherence and accuracy. But as with any large pull request, the responsibility for any bugs lies with me, and despite Jeremy's best efforts, I've been known to write some profoundly clever bugs. Thank you, Jeremy, for contributing your technical expertise to this book.

I don't know how much editing my writing required compared to the average author, but judging by the red markup on my drafts, Frances Saux is a saint. She shepherded me through this process and was an outstanding advocate for the reader. I could rely on her to keep my writing conversational and ask pointed questions that prompted me to elaborate on topics I take for granted. Thank you, Frances, for your patience and consistency throughout the writing process. This book certainly wouldn't be the same without your extensive efforts.

I would also like to thank Bill Pollock for giving this book his blessing; Barbara Yien for supervising it; Sharon Wilkey and Kate Kaminski for their copyediting and production expertise, respectively; Paula Fleming for proofreading; Derek Yee for the book's cover and interior design; Gina Redman for the cover illustration; and Matt Holt for reviewing Chapter 10 for technical accuracy.

Most of all, I'd like to thank my wife, Amandalyn; my son, Benjamin; and my daughter, Lilyanna. As with any extracurricular endeavor, the research and writing process consumed a lot of our family time. But I'm thankful for your patience while I strived to find a balance that worked for our family during this undertaking. Your love and support allowed me to step outside my comfort zone. Hopefully, my example will encourage you to do the same.

INTRODUCTION

With the advent of the internet came an ever-increasing demand for network engineers and developers. Today, personal computers, tablets, phones, televisions, watches, gaming systems, vehicles, common household items, and even doorbells communicate over the internet. Network programming makes all this possible. And *secure* network programming makes it trustworthy, driving increasing numbers of people to adopt these services. This book will teach you how to write contemporary network software using Go's asynchronous features.

Google created the Go programming language in 2007 to increase the productivity of developers working with large code bases. Since then, Go has earned a reputation as a fast, efficient, and safe language for the development and deployment of software at some of the largest companies in the world. Go is easy to learn and has a rich standard library, well suited for taking advantage of multicore, networked systems.

This book details the basics of network programming with an emphasis on security. You will learn socket-level programming including TCP, UDP, and Unix sockets, interact with application-level protocols like HTTPS and HTTP/2, serialize data with formats like Gob, JSON, XML, and protocol buffers, perform authentication and authorization for your network services, create streams and asynchronous data transfers, write gRPC microservices, perform structured logging and instrumentation, and deploy your applications to the cloud.

At the end of our journey, you should feel comfortable using Go, its standard library, and popular third-party packages to design and implement secure network applications and microservices. Every chapter uses best practices and includes nuggets of wisdom that will help you avoid potential pitfalls.

Who This Book Is For

If you'd like to learn how to securely share data over a network using standard protocols, all the while writing Go code that is stable, secure, and effective, this book is for you.

The target reader is a security-conscious developer or system administrator who wishes to take a deep dive into network programming and has a working knowledge of Go and Go's module support. That said, the first few chapters introduce basic networking concepts, so networking newcomers are welcome.

Staying abreast of contemporary protocols, standards, and best practices when designing and developing network applications can be difficult. That's why, as you work through this book, you'll be given increased responsibility. You'll also be introduced to tools and tech that will make your workload manageable.

Installing Go

To follow along with the code in this book, install the latest stable version of Go available at *https://golang.org/*. For most programs in this book, you'll need at least Go 1.12. That said, certain programs in this book are compatible with only Go 1.14 or newer. The book calls out the use of this code.

Keep in mind that the Go version available in your operating system's package manager may be several versions behind the latest stable version.

Recommended Development Environments

The code samples in this book are mostly compatible with Windows 10, Windows Subsystem for Linux, macOS Catalina, and contemporary Linux distributions, such as Ubuntu 20.04, Fedora 32, and Manjaro 20.1. The book calls out any code samples that are incompatible with any of those operating systems.

Some command line utilities used to test network services, such as `curl` or `nmap`, may not be part of your operating system's standard installation. You may need to install some of these command line utilities by using a package manager compatible with your operating system, such as Homebrew at *https://brew.sh/* for macOS or Chocolatey at *https://chocolatey.org/* for Windows 10. Contemporary Linux operating systems should include newer binaries in their package managers that will allow you to work through the code examples.

What's in This Book

This book is divided into four parts. In the first, you'll learn the foundational networking knowledge you'll need to understand before you begin writing network software.

Chapter 1: An Overview of Networked Systems introduces computer network organization models and the concepts of bandwidth, latency, network layers, and data encapsulation.

Chapter 2: Resource Location and Traffic Routing teaches you how human-readable names identify network resources, how devices locate network resources using their addresses, and how traffic gets routed between nodes on a network.

Part II of this book will put your new networking knowledge to use and teach you how to write programs that communicate using TCP, UDP, and Unix sockets. These protocols allow different devices to exchange data over a network and are fundamental to most network software you'll encounter or write.

Chapter 3: Reliable TCP Data Streams takes a deeper dive into the Transmission Control Protocol's handshake process, as well as its packet sequence numbering, acknowledgments, retransmissions, and other features that ensure reliable data transmission. You will use Go to establish and communicate over TCP sessions.

Chapter 4: Sending TCP Data details several programming techniques for transmitting data over a network using TCP, proxying data between network connections, monitoring network traffic, and avoiding common connection-handling bugs.

Chapter 5: Unreliable UDP Communication introduces you to the User Datagram Protocol, contrasting it with TCP. You'll learn how the difference between the two translates to your code and when to use UDP in your network applications. You'll write code that exchanges data with services using UDP.

Chapter 6: Ensuring UDP Reliability walks you through a practical example of performing reliable data transfers over a network using UDP.

Chapter 7: Unix Domain Sockets shows you how to efficiently exchange data between network services running on the same node using file-based communication.

The book's third part teaches you about application-level protocols such as HTTP and HTTP/2. You'll learn how to build applications that securely interact with servers, clients, and APIs over a network using TLS.

Chapter 8: Writing HTTP Clients uses Go's excellent HTTP client to send requests to, and receive resources from, servers over the World Wide Web.

Chapter 9: Building HTTP Services demonstrates how to use handlers, middleware, and multiplexers to build capable HTTP-based applications with little code.

Chapter 10: Caddy: A Contemporary Web Server introduces you to a contemporary web server named Caddy that offers security, performance, and extensibility through modules and configuration adapters.

Chapter 11: Securing Communications with TLS gives you the tools to incorporate authentication and encryption into your applications using TLS, including mutual authentication between a client and a server.

Part IV shows you how to serialize data into formats suitable for exchange over a network; gain insight into your services; and deploy your code to Amazon Web Services, Google Cloud, and Microsoft Azure.

Chapter 12: Data Serialization discusses how to exchange data between applications that use different platforms and languages. You'll write programs that serialize and deserialize data using Gob, JSON, and protocol buffers and communicate using gRPC.

Chapter 13: Logging and Metrics introduces tools that provide insight into how your services are working, allowing you to proactively address potential problems and recover from failures.

Chapter 14: Moving to the Cloud discusses how to develop and deploy a serverless application on Amazon Web Services, Google Cloud, and Microsoft Azure.

PART I

NETWORK ARCHITECTURE

1

AN OVERVIEW OF NETWORKED SYSTEMS

In the digital age, an increasing number of devices communicate over computer networks. A *computer network* is a connection between two or more devices, or *nodes*, that allows each node to share data. These connections aren't inherently reliable or secure. Thankfully, Go's standard library and its rich ecosystem are well suited for writing secure, reliable network applications.

This chapter will give you the foundational knowledge needed for this book's exercises. You'll learn about the structure of networks and how networks use protocols to communicate.

Choosing a Network Topology

The organization of nodes in a network is called its *topology*. A network's topology can be as simple as a single connection between two nodes or as

complex as a layout of nodes that don't share a direct connection but are nonetheless able to exchange data. That's generally the case for connections between your computer and nodes on the internet. Topology types fall into six basic categories: point-to-point, daisy chain, bus, ring, star, and mesh.

In the simplest network, *point-to-point*, two nodes share a single connection (Figure 1-1). This type of network connection is uncommon, though it is useful when direct communication is required between two nodes.

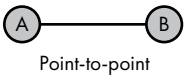

Figure 1-1: A direct connection between two nodes

A series of point-to-point connections creates a *daisy chain*. In the daisy chain in Figure 1-2, traffic from node C, destined for node F, must traverse nodes D and E. Intermediate nodes between an origin node and a destination node are commonly known as *hops*. You are unlikely to encounter this topology in a modern network.

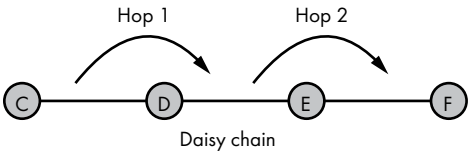

Figure 1-2: Point-to-point segments joined in a daisy chain

Bus topology nodes share a common network link. Wired bus networks aren't common, but this type of topology drives wireless networks. The nodes on a wired network see all the traffic and selectively ignore or accept it, depending on whether the traffic is intended for them. When node H sends traffic to node L in the bus diagram in Figure 1-3, nodes I, J, K, and M receive the traffic but ignore it. Only node L accepts the data because it's the intended recipient. Although wireless clients can see each other's traffic, traffic is usually encrypted.

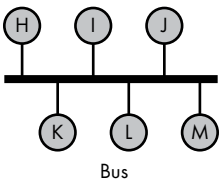

Figure 1-3: Nodes connected in a bus topology

A *ring* topology, which was used in some fiber-optic network deployments, is a closed loop in which data travels in a single direction. In Figure 1-4, for example, node N could send a message destined for node R by way of nodes O, P, and Q. Nodes O, P, and Q retransmit the message until it reaches node R. If node P fails to retransmit the message, it will never reach its destination. Because of this design, the slowest node can limit the speed at which data travels. Assuming traffic travels clockwise and node Q is the slowest, node Q slows traffic sent from node O to node N. However, traffic sent from node N to node O is not limited by node Q's slow speed since that traffic does not traverse node Q.

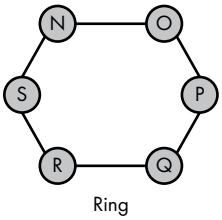

Figure 1-4: Nodes arranged in a ring, with traffic traveling in a single direction

In a *star* topology, a central node has individual point-to-point connections to all other nodes. You will likely encounter this network topology in wired networks. The central node, as shown in Figure 1-5, is often a *network switch*, which is a device that accepts data from the origin nodes and retransmits data to the destination nodes, like a postal service. Adding nodes is a simple matter of connecting them to the switch. Data can traverse only a single hop within this topology.

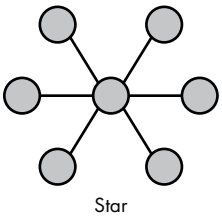

Figure 1-5: Nodes connected to a central node, which handles traffic between nodes

Every node in a fully connected *mesh* network has a direct connection to every other node (Figure 1-6). This topology eliminates single points of failure because the failure of a single node doesn't affect traffic between any other nodes on the network. On the other hand, costs and complexity increase as the number of nodes increases, making this topology untenable for large-scale networks. This is another topology you may encounter only in larger wireless networks.

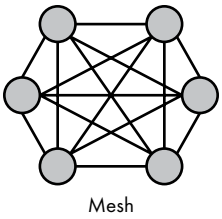

Figure 1-6: Interconnected nodes in a mesh network

You can also create a hybrid network topology by combining two or more basic topologies. Real-world networks are rarely composed of just one network topology. Rather, you are likely to encounter hybrid topologies. Figure 1-7 shows two examples. The *star-ring* hybrid network is a series of ring networks connected to a central node. The *star-bus* hybrid network is a hierarchical topology formed by the combination of bus and star network topologies.

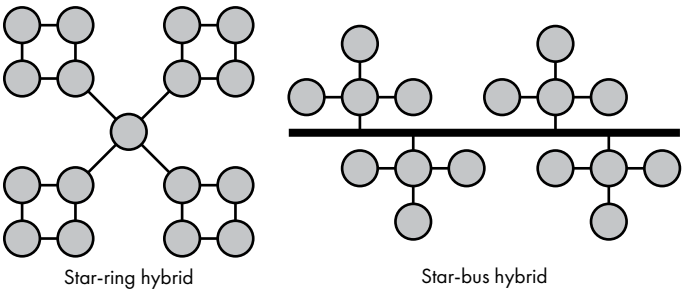

Figure 1-7: The star-ring and star-bus hybrid topologies

Hybrid topologies are meant to improve reliability, scalability, and flexibility by taking advantage of each topology's strengths and by limiting the disadvantages of each topology to individual network segments.

For example, the failure of the central node in the *star-ring* hybrid in Figure 1-7 would affect inter-ring communication only. Each ring network would continue to function normally despite its isolation from the other rings. The failure of a single node in a ring would be much easier to diagnose in a star-ring hybrid network than in a single large ring network. Also, the outage would affect only a subset of the overall network.

Bandwidth vs. Latency

Network *bandwidth* is the amount of data we can send over a network connection in an interval of time. If your internet connection is advertised as

100Mbps download, that means your internet connection should theoretically be able to transfer up to 100 megabits every second from your internet service provider (ISP) to your modem.

ISPs inundate us with advertisements about the amount of bandwidth they offer, so much so that it's easy for us to fixate on bandwidth and equate it with the speed of the connection. However, faster doesn't always mean greater performance. It may seem counterintuitive, but a lower-bandwidth network connection may seem to have better performance than a higher-bandwidth network connection because of one characteristic: latency.

Network *latency* is a measure of the time that passes between sending a network resource request and receiving a response. An example of latency is the delay that occurs between clicking a link on a website and the site's rendering the resulting page. You've probably experienced the frustration of clicking a link that fails to load before your web browser gives up on ever receiving a reply from the server. This happens when the latency is greater than the maximum amount of time your browser will wait for a reply.

High latency can negatively impact the user experience, lead to attacks that make your service inaccessible to its users, and drive users away from your software or service. The importance of managing latency in network software is often underappreciated by software developers. Don't fall into the trap of thinking that bandwidth is all that matters for optimal network performance.

A website's latency comes from several sources: the network latency between the client and server, the time it takes to retrieve data from a data store, the time it takes to compile dynamic content on the server side, and the time it takes for the web browser to render the page. If a user clicks a link and the page takes too long to render, the user likely won't stick around for the results, and the latency will drive traffic away from your application. Keeping latency to a minimum while writing network software, be it web applications or application-programming interfaces, will pay dividends by improving the user experience and your application's ranking in popular search engines.

You can address the most common sources of latency in several ways. First, you can reduce both the distance and the number of hops between users and your service by using a content delivery network (CDN) or cloud infrastructure to locate your service near your users. Optimizing the request and response sizes will further reduce latency. Incorporating a caching strategy in your network applications can have dramatic effects on performance. Finally, taking advantage of Go's concurrency to minimize server-side blocking of the response can help. We'll focus on this in the later chapters of this book.

The Open Systems Interconnection Reference Model

In the 1970s, as computer networks became increasingly complex, researchers created the *Open Systems Interconnection (OSI) reference model* to

standardize networking. The OSI reference model serves as a framework for the development of and communication about protocols. *Protocols* are rules and procedures that determine the format and order of data sent over a network. For example, communication using the *Transmission Control Protocol (TCP)* requires the recipient of a message to reply with an acknowledgment of receipt. Otherwise, TCP may retransmit the message.

Although OSI is less relevant today than it once was, it's still important to be familiar with it so you'll understand common concepts, such as lower-level networking and routing, especially with respect to the involved hardware.

The Hierarchal Layers of the OSI Reference Model

The OSI reference model divides all network activities into a strict hierarchy composed of seven layers. Visual representations of the OSI reference model, like the one in Figure 1-8, arrange the layers into a stack, with Layer 7 at the top and Layer 1 at the bottom.

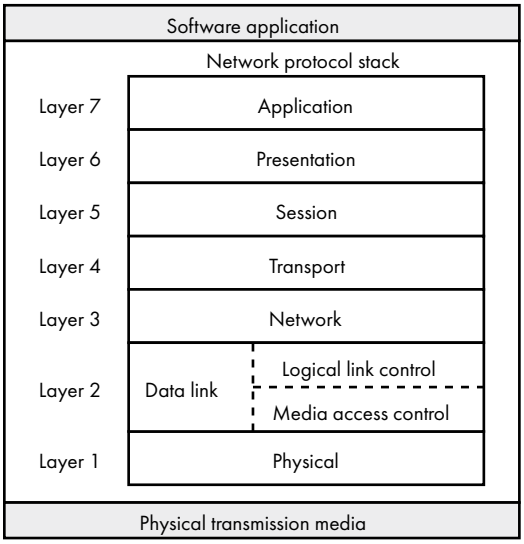

Figure 1-8: Seven layers of the OSI reference model

It's easy to interpret these layer designations as independent units of code. Rather, they describe abstractions we ascribe to parts of our software. For example, there is no *Layer 7* library you can incorporate into your software. But you can say that the software you wrote implements a service at Layer 7. The seven layers of the OSI model are as follows:

Layer 7—application layer Your network applications and libraries most often interact with the application layer, which is responsible for identifying hosts and retrieving resources. Web browsers, Skype, and bit torrent clients are examples of Layer 7 applications.

Layer 6—presentation layer The presentation layer prepares data for the network layer when that data is moving down the stack, and it presents data to the application layer when that data moves up the stack. Encryption, decryption, and data encoding are examples of Layer 6 functions.

Layer 5—session layer The session layer manages the connection life cycle between nodes on a network. It's responsible for establishing the connection, managing connection time-outs, coordinating the mode of operation, and terminating the connection. Some Layer 7 protocols rely on services provided by Layer 5.

Layer 4—transport layer The transport layer controls and coordinates the transfer of data between two nodes while maintaining the reliability of the transfer. Maintaining the reliability of the transfer includes correcting errors, controlling the speed of data transfer, chunking or segmenting the data, retransmitting missing data, and acknowledging received data. Often protocols in this layer might retransmit data if the recipient doesn't acknowledge receipt of the data.

Layer 3—network layer The network layer is responsible for transmitting data between nodes. It allows you to send data to a network address without having a direct point-to-point connection to the remote node. OSI does not require protocols in this layer to provide reliable transport or report transmission errors to the sender. The network layer is home to network management protocols involved in routing, addressing, multicasting, and traffic control. The next chapter covers these topics.

Layer 2—data link layer The data link layer handles data transfers between two directly connected nodes. For example, the data link layer facilitates data transfer from a computer to a switch and from the switch to another computer. Protocols in this layer identify and attempt to correct errors on the physical layer.

The data link layer's retransmission and flow control functions are dependent on the underlying physical medium. For example, Ethernet does not retransmit incorrect data, whereas wireless does. This is because bit errors on Ethernet networks are infrequent, whereas they're common over wireless. Protocols further up the network protocol stack can ensure that the data transmission is reliable if this layer doesn't do so, though generally with less efficiency.

Layer 1—physical layer The physical layer converts bits from the network stack to electrical, optic, or radio signals suitable for the underlying physical medium and from the physical medium back into bits. This layer controls the bit rate. The bit rate is the data speed limit. A gigabit per second bit rate means data can travel at a maximum of 1 billion bits per second between the origin and destination.

A common confusion when discussing network transmission rates is using bytes per second instead of bits per second. We count the number of zeros and ones, or *bits*, we can transfer per second. Therefore, network transmission rates are measured in bits per second. We use bytes per second when discussing the amount of data transferred.

If your ISP advertises a 100Mbps download rate, that doesn't mean you can download a 100MB file in one second. Rather, it may take closer to eight seconds under ideal network conditions. It's appropriate to say we can transfer a maximum of 12.5MB per second over the 100Mbps connection.

Sending Traffic by Using Data Encapsulation

Encapsulation is a method of hiding implementation details or making only relevant details available to the recipient. Think of encapsulation as being like a package you send through the postal service. We could say that the envelope encapsulates its contents. In doing so, it may include the destination address or other crucial details used by the next leg of its journey. The actual contents of your package are irrelevant; only the details on the package are important for transport.

As data travels down the stack, it's encapsulated by the layer below. We typically call the data traveling down the stack a *payload*, although you might see it referred to as a *message body*. The literature uses the term *service data unit (SDU)*. For example, the transport layer encapsulates payloads from the session layer, which in turn encapsulates payloads from the presentation layer. When the payload moves up the stack, each layer strips the header information from the previous stack.

Even protocols that operate in a single OSI layer use data encapsulation. Take version 1 of the *HyperText Transfer Protocol (HTTP/1)*, for example, a Layer 7 protocol that both the client and the server use to exchange web content. HTTP defines a complete message, including header information, that the client sends from its Layer 7 to the server's Layer 7; the network stack delivers the client's request to the HTTP server application. The HTTP server application initiates a response to its network stack, which creates a Layer 7 payload and sends it back to the client's Layer 7 application (Figure 1-9).

Communication between the client and the server on the same layer is called *horizontal communication*, a term that makes it sound like a single-layer protocol on the client directly communicates with its counterpart on the server. In fact, in horizontal communication, data must travel all the way down the client's stack, then back up the server's stack.

For example, Figure 1-10 shows how an HTTP request traverses the stack.

Generally, a payload travels down the client's network stack, over physical media to the server, and up the server's network stack to its corresponding layer. The result is that data sent from one layer at the origin node arrives at the same layer on the destination node. The server's response takes the same path in the opposite direction. On the client's side, Layer 6 receives Layer 7's

payload, then encapsulates the payload with a header to create Layer 6's payload. Layer 5 receives Layer 6's payload, adds its own header, and sends its payload on to Layer 4, where we're introduced to our first transmission protocol: TCP.

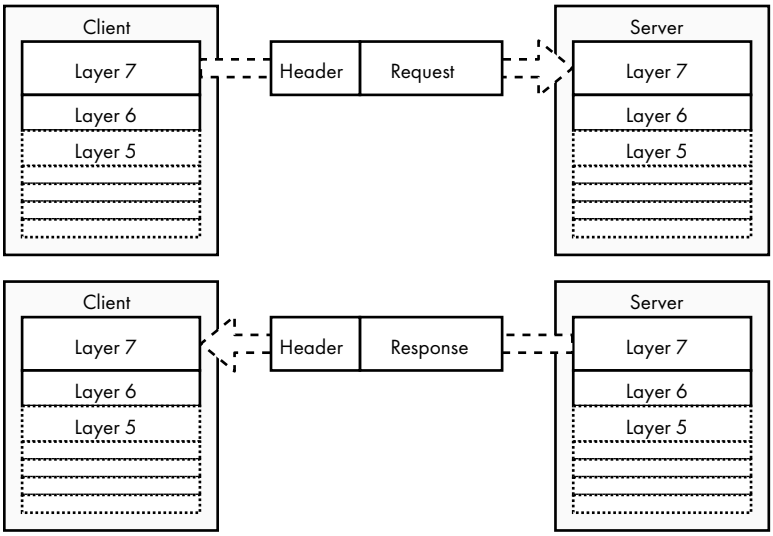

Figure 1-9: Horizontal communication from the client to the server and back

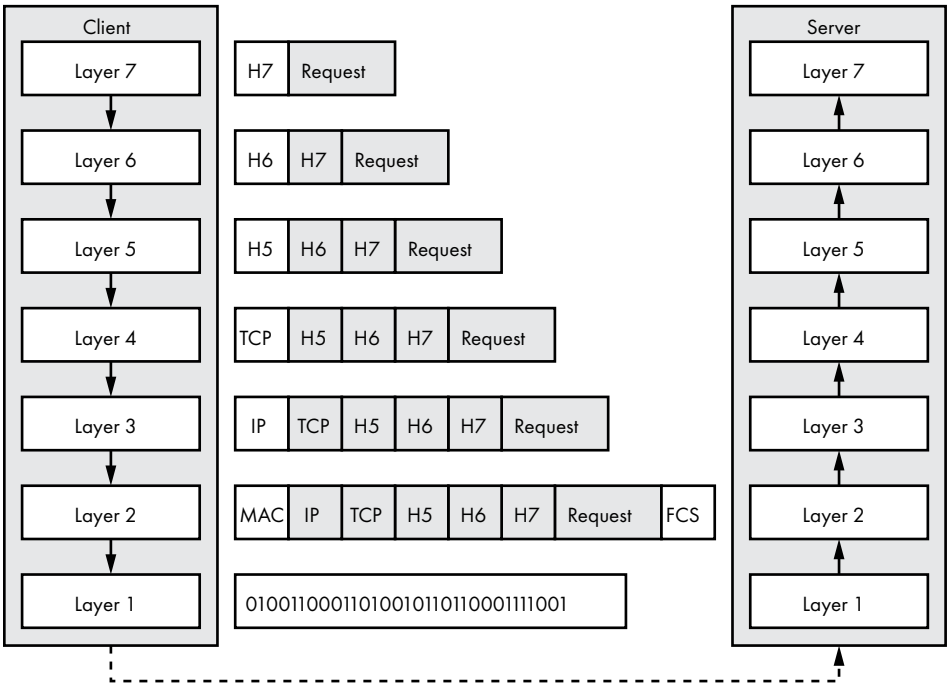

Figure 1-10: An HTTP request traveling from Layer 7 on the client to Layer 7 on the server

TCP is a Layer 4 protocol whose payloads are also known as *segments* or *datagrams*. TCP accepts Layer 5's payload and adds its header before sending the segment on to Layer 3. The *Internet Protocol (IP)* at Layer 3 receives the TCP segment and encapsulates it with a header to create Layer 3's payload, which is known as a *packet*. Layer 2 accepts the packet and adds a header and a footer, creating its payload, called a *frame*. Layer 2's header translates the recipient's IP address into a *media access control (MAC)* address, which is a unique identifier assigned to the node's network interface. Its footer contains a *frame check sequence (FCS)*, which is a checksum to facilitate error detection. Layer 1 receives Layer 2's payload in the form of bits and sends the bits to the server.

The server's Layer 1 receives the bits, converts them to a frame, and sends the frame up to Layer 2. Layer 2 strips its header and footer from the frame and passes the packet up to Layer 3. The process of reversing each layer's encapsulation continues up the stack until the payload reaches Layer 7. Finally, the HTTP server receives the client's request from the network stack.

The TCP/IP Model

Around the same time as researchers were developing the OSI reference model, the Defense Advanced Research Projects Agency (DARPA), an agency of the US Department of Defense, spearheaded a parallel effort to develop protocols. This effort resulted in a set of protocols we now call the *TCP/IP model*. The project's impact on the US military, and subsequently the world's communications, was profound. The TCP/IP model reached critical mass when Microsoft incorporated it into Windows 95 in the early 1990s. Today, TCP/IP is ubiquitous in computer networking, and it's the protocol stack we'll use in this book.

TCP/IP—named for the Transmission Control Protocol and the Internet Protocol—facilitated networks designed using the *end-to-end principle*, whereby each network segment includes only enough functionality to properly transmit and route bits; all other functionality belongs to the endpoints, or the sender and receiver's network stacks. Contrast this with modern cellular networks, where more of the network functionality must be provided by the network between cell phones to allow for a cell phone connection to jump from tower to tower without disconnecting its phone call. The TCP/IP specification recommends that implementations be robust; they should send well-formed packets yet accept any packet whose intention is clear, regardless of whether the packet adheres to technical specifications.

Like the OSI reference model, TCP/IP relies on layer encapsulation to abstract functionality. It consists of four named layers: the application, transport, internet, and link layers. TCP/IP's application and link layers generalize their OSI counterparts, as shown in Figure 1-11.

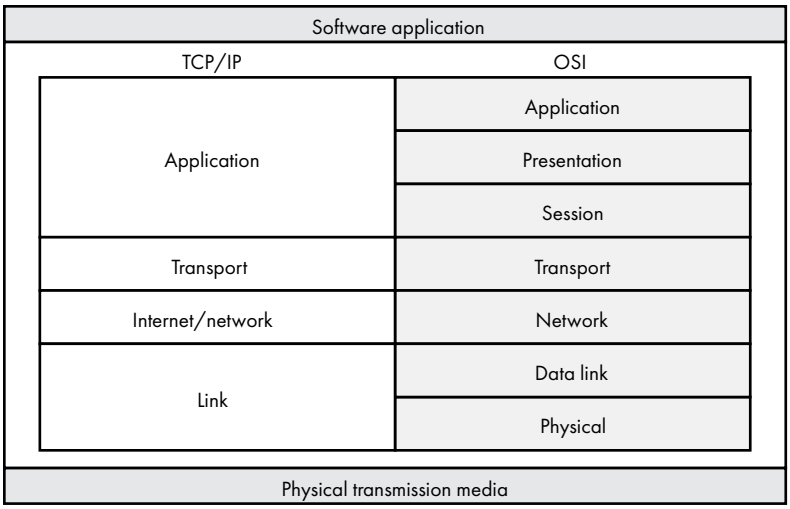

Figure 1-11: The four-layer TCP/IP model compared to the seven-layer OSI reference model

The TCP/IP model simplifies OSI's application, presentation, and session layers into a single application layer, primarily because TCP/IP's protocols frequently cross boundaries of OSI Layers 5 through 7. Likewise, OSI's data link and physical layers correspond to TCP/IP's link layer. TCP/IP's and OSI's transport and network layers share a one-to-one relationship.

This simplification exists because researchers developed prototype implementations first and then formally standardized their final implementation, resulting in a model geared toward practical use. On the other hand, committees spent considerable time devising the OSI reference model to address a wide range of requirements before anyone created an implementation, leading to the model's increased complexity.

The Application Layer

Like OSI's application layer, the TCP/IP model's *application layer* interacts directly with software applications. Most of the software we write uses protocols in this layer, and when your web browser retrieves a web page, it reads from this layer of the stack.

You'll notice that the TCP/IP application layer encompasses three OSI layers. This is because TCP/IP doesn't define specific presentation or session functions. Instead, the specific application protocol implementations concern themselves with those details. As you'll see, some TCP/IP application layer protocols would be hard-pressed to fit neatly into a single upper layer of the OSI model, because they have functionality that spans more than one OSI layer.

Common TCP/IP application layer protocols include HTTP, the *File Transfer Protocol (FTP)* for file transfers between nodes, and the *Simple Mail Transfer Protocol (SMTP)* for sending email to mail servers. The

Dynamic Host Configuration Protocol (DHCP) and the *Domain Name System (DNS)* also function in the application layer. DHCP and DNS provide the addressing and name resolution services, respectively, that allow other application layer protocols to operate. HTTP, FTP, and SMTP are examples of protocol implementations that provide the presentation or session functionality in TCP/IP's application layer. We'll discuss these protocols in later chapters.

The Transport Layer

Transport layer protocols handle the transfer of data between two nodes, like OSI's Layer 4. These protocols can help ensure *data integrity* by making sure that all data sent from the origin completely and correctly makes its way to the destination. Keep in mind that data integrity doesn't mean the destination will receive all segments we send through the transport layer. There are just too many causes of packet loss over a network. It does mean that TCP specifically will make sure the data received by the destination is in the correct order, without duplicate data or missing data.

The primary transport layer protocols you'll use in this book are TCP and the *User Datagram Protocol (UDP)*. As mentioned in "Sending Traffic by Using Data Encapsulation" on page 10, this layer handles segments, or datagrams.

NOTE *TCP also overlaps a bit with OSI's Layer 5, because TCP includes session-handling capabilities that would otherwise fall under the scope of OSI's session layer. But, for our purposes, it's okay to generalize the transport layer as representing OSI's Layer 4.*

Most of our network applications rely on the transport layer protocols to handle the error correction, flow control, retransmission, and transport acknowledgment of each segment. However, the TCP/IP model doesn't require every transport layer protocol to fulfill each of those elements. UDP is one such example. If your application requires the use of UDP for maximal throughput, the onus is on you to implement some sort of error checking or session management, since UDP provides neither.

The Internet Layer

The *internet layer* is responsible for routing packets of data from the upper layers between the origin node and the destination node, often over multiple networks with heterogeneous physical media. It has the same functions as OSI's Layer 3 network layer. (Some sources may refer to TCP/IP's internet layer as the *network layer*.)

Internet Protocol version 4 (IPv4), *Internet Protocol version 6 (IPv6)*, *Border Gateway Protocol (BGP)*, *Internet Control Message Protocol (ICMP)*, *Internet Group Management Protocol (IGMP)*, and the *Internet Protocol Security (IPsec)* suite, among others, provide host identification and routing to TCP/IP's internet layer. We will cover these protocols in the next chapter, when we discuss

host addressing and routing. For now, know that this layer plays an integral role in ensuring that the data we send reaches its destination, no matter the complexity between the origin and destination.

The Link Layer

The *link layer*, which corresponds to Layers 1 and 2 of the OSI reference model, is the interface between the core TCP/IP protocols and the physical media.

The link layer's *Address Resolution Protocol (ARP)* translates a node's IP address to the MAC address of its network interface. The link layer embeds the MAC address in each frame's header before passing the frame on to the physical network. We'll discuss MAC addresses and their routing significance in the next chapter.

Not all TCP/IP implementations include link layer protocols. Older readers may remember the joys of connecting to the internet over phone lines using analog modems. Analog modems made serial connections to ISPs. These serial connections didn't include link layer support via the serial driver or modem. Instead, they required the use of link layer protocols, such as the *Serial Line Internet Protocol (SLIP)* or the *Point-to-Point Protocol (PPP)*, to fill the void. Those that do not implement a link layer typically rely on the underlying network hardware and device drivers to pick up the slack. The TCP/IP implementations over Ethernet, wireless, and fiber-optic networks we'll use in this book rely on device drivers or network hardware to fulfill the link layer portion of their TCP/IP stack.

What You've Learned

In this chapter, you learned about common network topologies and how to combine those topologies to maximize their advantages and minimize their disadvantages. You also learned about the OSI and TCP/IP reference models, their layering, and data encapsulation. You should feel comfortable with the order of each layer and how data moves from one layer to the next. Finally, you learned about each layer's function and the role it plays in sending and receiving data between nodes on a network.

This chapter's goal was to give you enough networking knowledge to make sense of the next chapter. However, it's important that you explore these topics in greater depth, because comprehensive knowledge of networking principles and architectures can help you devise better algorithms. I'll suggest additional reading for each of the major topics covered in this chapter to get you started. I also recommend you revisit this chapter after working through some of the examples in this book.

The OSI reference model is available for reading online at *https://www.itu.int/rec/T-REC-X.200-199407-I/en/*. Two Requests for Comments (RFCs)—detailed publications meant to describe internet technologies—outline the TCP/IP reference model: RFC 1122 and RFC 1123 (*https://tools.ietf.org/html/rfc1122/* and *https://tools.ietf.org/html/rfc1123/*). RFC 1122 covers the lower

three layers of the TCP/IP model, whereas RFC 1123 describes the application layer and support protocols, such as DNS. If you'd like a more comprehensive reference for the TCP/IP model, you'd be hard-pressed to do better than *The TCP/IP Guide* by Charles M. Kozierok (No Starch Press, 2005).

Network latency has plagued countless network applications and spawned an industry. Some CDN providers write prolifically on the topic of latency and interesting issues they've encountered while improving their offerings. CDN blogs that provide insightful posts include the Cloudflare Blog (*https://blog.cloudflare.com/*), the KeyCDN Blog (*https://www.keycdn.com/blog/*), and the Fastly Blog (*https://www.fastly.com/blog/*). If you're purely interested in learning more about latency and its sources, consider "Latency (engineering)" on Wikipedia (*https://en.wikipedia.org/wiki/Latency_(engineering)*) and Cloudflare's glossary (*https://www.cloudflare.com/learning/performance/glossary/what-is-latency/*) as starting points.

2

RESOURCE LOCATION AND TRAFFIC ROUTING

To write effective network programs, you need to understand how to use human-readable names to identify nodes on the internet, how those names are translated into addresses for network devices to use, and how traffic makes its way between nodes on the internet, even if they're on opposite sides of the planet. This chapter covers those topics and more.

We'll first have a look at how IP addresses identify hosts on a network. Then we'll discuss *routing*, or sending traffic between network hosts that aren't directly connected, and cover some common routing protocols. Finally, we'll discuss *domain name resolution* (the process of translating human-readable names to IP addresses), potential privacy implications of DNS, and the solutions to overcome those privacy concerns.

You'll need to understand these topics to provide comprehensive network services and locate the resources used by your services, such as third-party application programming interfaces (APIs). This information should

also help you debug inevitable network outages or performance issues your code may encounter. For example, say you provide a service that integrates the Google Maps API to provide interactive maps and navigation. Your service would need to properly locate the API endpoint and route traffic to it. Or your service may need to store archives in an Amazon Simple Storage Service (S3) bucket via the Amazon S3 API. In each example, name resolution and routing play an integral role.

The Internet Protocol

The *Internet Protocol (IP)* is a set of rules that dictate the format of data sent over a network—specifically, the internet. *IP addresses* identify nodes on a network at the internet layer of the TCP/IP stack, and you use them to facilitate communication between nodes.

IP addresses function in the same way as postal addresses; nodes send packets to other nodes by addressing packets to the destination node's IP address. Just as it's customary to include a return address on postal mail, packet headers include the IP address of the origin node as well. Some protocols require an acknowledgment of successful delivery, and the destination node can use the origin node's IP address to send the delivery confirmation.

Two versions of IP addresses are in public use: IPv4 and IPv6. This chapter covers both.

IPv4 Addressing

IPv4 is the fourth version of the Internet Protocol. It was the first IP version in use on the internet's precursor, ARPANET, in 1983, and the predominant version in use today. IPv4 addresses are 32-bit numbers arranged in four groups of 8 bits (called *octets*) separated by decimal points.

NOTE *RFCs use the term* octet *as a disambiguation of the term* byte*, because a byte's storage size has historically been platform dependent.*

The total range of 32-bit numbers limits us to just over four billion possible IPv4 addresses. Figure 2-1 shows the binary and decimal representation of an IPv4 address.

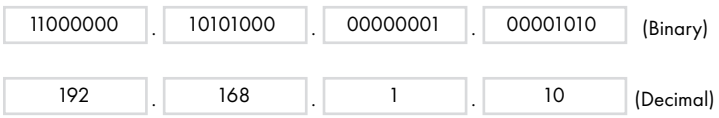

Figure 2-1: Four 8-bit octets representing an IPv4 address in both binary and decimal formats

The first line of Figure 2-1 illustrates an IPv4 address in binary form. The second line is the IPv4 address's decimal equivalent. We usually write IPv4 addresses in the more readable decimal format when displaying them or when using them in code. We will use their binary representation when we're discussing network addressing later in this section.

Network and Host IDs

The 32 bits that compose an IPv4 address represent two components: a network ID and a host ID. The *network ID* informs the network devices responsible for shuttling packets toward their destination about the next appropriate hop in the transmission. These devices are called *routers*. Routers are like the mail carrier of a network, in that they accept data from a device, examine the network ID of the destination address, and determine where the data needs to be sent to reach its destination. You can think of the network ID as a mailing address's ZIP (or postal) code.

Once the data reaches the destination network, the router uses the *host ID* to deliver the data to the specific recipient. The host ID is like your street address. In other words, a network ID identifies a group of nodes whose address is part of the same network. We'll see what network and host IDs look like later in this chapter, but Figure 2-2 shows IPv4 addresses sharing the same network ID.

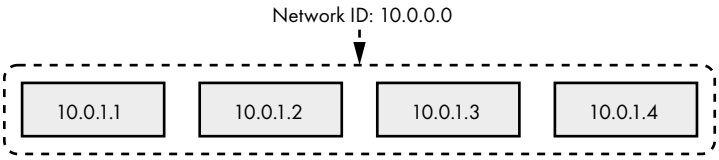

Figure 2-2: A group of nodes sharing the same network ID

Figure 2-3 shows the breakdown of common network ID and host ID sizes in a 32-bit IPv4 address.

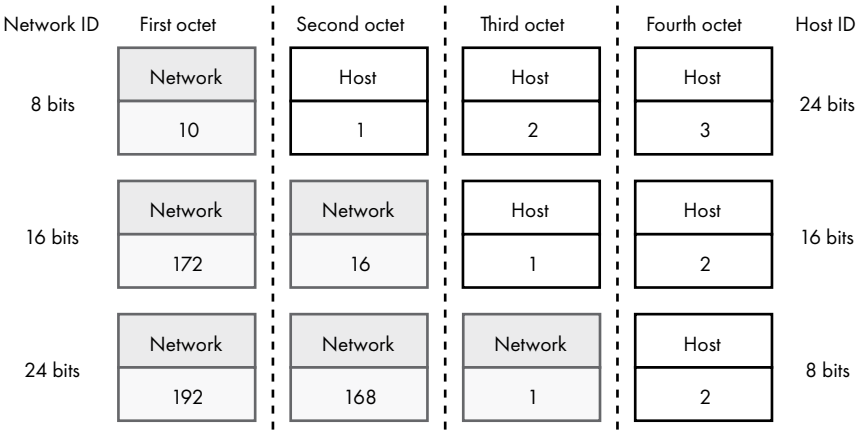

Figure 2-3: Common network ID and host ID sizes

The network ID portion of an IPv4 address always starts with the leftmost bit, and its size is determined by the size of the network it belongs to. The remaining bits designate the host ID. For example, the first 8 bits of the IPv4 address represent the network ID in an 8-bit network, and the remaining 24 bits represent the host ID.

Figure 2-4 shows the IP address 192.168.156.97 divided into its network ID and host ID. This IP address is part of a 16-bit network. This tells us that the first 16 bits form the network ID and the remaining 16 bits form the host ID.

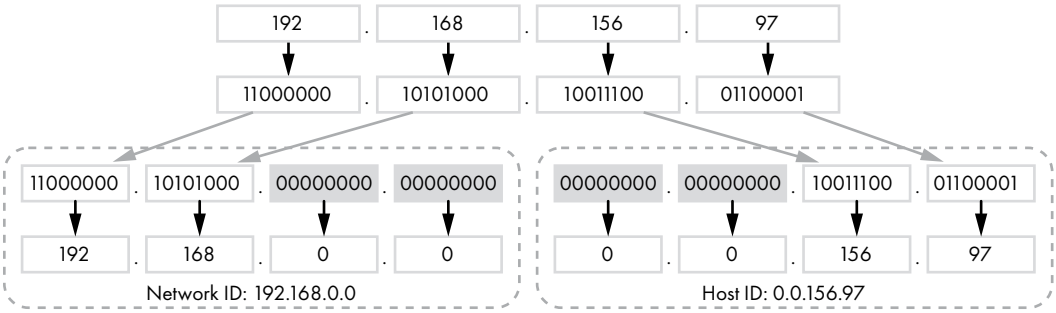

Figure 2-4: Deriving the network ID and the host ID from an IPv4 address in a 16-bit network

To derive the network ID for this example, you take the first 16 bits and append zeros for the remaining bits to produce the 32-bit network ID of 192.168.0.0. You prepend zeroed bits to the last 16 bits, resulting in the 32-bit host ID of 0.0.156.97.

Subdividing IPv4 Addresses into Subnets

IPv4's network and host IDs allow you to *subdivide*, or partition, the more than four billion IPv4 addresses into smaller groups to keep the network secure and easier to manage. All IP addresses in these smaller networks, called *subnets*, share the same network ID but have unique host IDs. The size of the network dictates the number of host IDs and, therefore, the number of individual IP addresses in the network.

Identifying individual networks allows you to control the flow of information between networks. For example, you could split your network into a subnet meant for public services and another for private services. You could then allow external traffic to reach your public services while preventing external traffic from reaching your private network. As another example, your bank provides services such as online banking, customer support, and mobile banking. These are public services that you interact with after successful authentication. But you don't have access to the bank's internal network, where its systems manage electronic transfers, balance ledgers, serve internal email, and so on. These services are restricted to the bank's employees via the private network.

Allocating Networks with CIDR

You allocate networks using a method known as *Classless Inter-Domain Routing (CIDR)*. In CIDR, you indicate the number of bits in the network ID by appending a *network prefix* to each IP address, consisting of a forward slash and an integer. Though it's appended to the end of the IP address,

you call it a *prefix* rather than a *suffix* because it indicates how many of the IP address's most significant bits, or prefixed bits, constitute the network ID. For example, you'd write the IP address 192.168.156.97 from Figure 2-4 as 192.168.156.97/16 in CIDR notation, indicating that it belongs to a 16-bit network and that the network ID is the first 16 bits of the IP address.

From there, you can derive the network IP address by applying a subnet mask. Subnet masks encode the CIDR network prefix in its decimal representation. They are applied to an IP address using a bitwise AND to derive the network ID.

Table 2-1 details the most common CIDR network prefixes, the corresponding subnet mask, the available networks for each network prefix, and the number of usable hosts in each network.

Table 2-1: CIDR Network Prefix Lengths and Their Corresponding Subnet Masks

CIDR network prefix length	Subnet mask	Available networks	Usable hosts per network
8	255.0.0.0	1	16,777,214
9	255.128.0.0	2	8,388,606
10	255.192.0.0	4	4,194,302
11	255.224.0.0	8	2,097,150
12	255.240.0.0	16	1,048,574
13	255.248.0.0	32	524,286
14	255.252.0.0	64	262,142
15	255.254.0.0	128	131,070
16	255.255.0.0	256	65,534
17	255.255.128.0	512	32,766
18	255.255.192.0	1,024	16,382
19	255.255.224.0	2,048	8,190
20	255.255.240.0	4,096	4,094
21	255.255.248.0	8,192	2,046
22	255.255.252.0	16,384	1,022
23	255.255.254.0	32,768	510
24	255.255.255.0	65,536	254
25	255.255.255.128	131,072	126
26	255.255.255.192	262,144	62
27	255.255.255.224	524,288	30
28	255.255.255.240	1,048,576	14
29	255.255.255.248	2,097,152	6
30	255.255.255.252	4,194,304	2

You may have noticed that the number of usable hosts per network is two less than expected in each row because each network has two special addresses. The first IP address in the network is the network address, and the last IP address is the broadcast address. (We'll cover broadcast addresses a bit later in this chapter.) Take 192.168.0.0/16, for example. The first IP address in the network is 192.168.0.0. This is the network address. The last IP address in the network is 192.168.255.255, which is the broadcast address. For now, understand that you do not assign the network IP address or the broadcast IP address to a host's network interface. These special IP addresses are used for routing data between networks and broadcasting, respectively.

The 31- and 32-bit network prefixes are purposefully absent from Table 2-1, largely because they are beyond the scope of this book. If you're curious about 31-bit network prefixes, RFC 3021 covers their application. A 32-bit network prefix signifies a single-host network. For example, 192.168.1.1/32 represents a subnetwork of one node with the address 192.168.1.1.

Allocating Networks That Don't Break at an Octet Boundary

Some network prefixes don't break at an octet boundary. For example, Figure 2-5 derives the network ID and host ID of 192.168.156.97 in a 19-bit network. The full IP address in CIDR notation is 192.168.156.97/19.

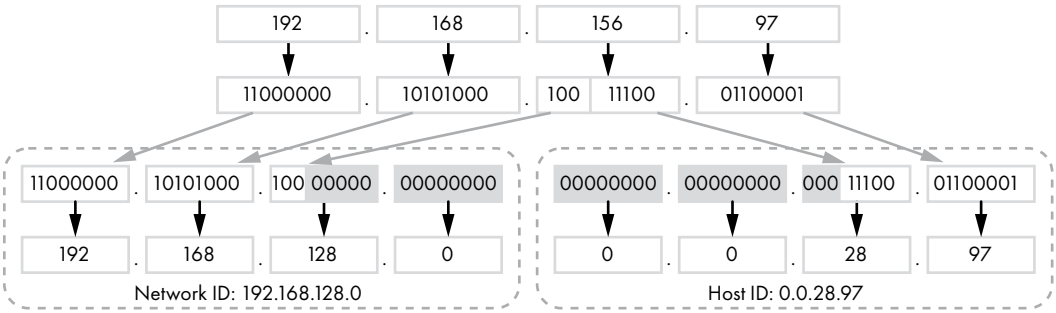

Figure 2-5: Deriving the network ID and the host ID from the IPv4 address in a 19-bit network

In this case, since the network prefix isn't a multiple of 8 bits, an octet's bits are split between the network ID and host ID. The 19-bit network example in Figure 2-5 results in the network ID of 192.168.128.0 and the host ID of 0.0.28.97, where the network ID borrows 3 bits from the third octet, leaving 13 bits for the host ID.

Appending a zeroed host ID to the network ID results in the network address. In a comparable manner, appending a host ID in which all its bits are 1 to the network ID derives the broadcast address. But the third octet's equaling 156 can be a little confusing. Let's focus on just the third octet. The third octet of the network ID is 1000 0000. The third octet of the host ID of all ones is 0001 1111 (the first 3 bits are part of the network ID, remember). If we append the network ID's third octet to the host ID's third octet, the result is 1001 1111, which is the decimal 156.

Private Address Spaces and Localhost

RFC 1918 details the private address spaces of 10.0.0.0/8, 172.16.0.0/12, and 192.168.0.0/16 for use in local networks. Universities, corporations, governments, and residential networks can use these subnets for local addressing.

In addition, each host has the 127.0.0.0/8 subnet designated as its local subnet. Addresses in this subnet are local to the host and simply called *localhost*. Even if your computer is not on a network, it should still have an address on the 127.0.0.0/8 subnet, most likely 127.0.0.1.

Ports and Socket Addresses

If your computer were able to communicate over the network with only one node at a time, that wouldn't provide a very efficient or pleasant experience. It would become annoying if your streaming music stopped every time you clicked a link in your web browser because the browser needed to interrupt the stream to retrieve the requested web page. Thankfully, TCP and UDP allow us to multiplex data transmissions by using *ports*.

The operating system uses ports to uniquely identify data transmission between nodes for the purposes of multiplexing the outgoing application data and demultiplexing the incoming data back to the proper application. The combination of an IP address and a port number is a *socket address*, typically written in the format *address:port*.

Ports are 16-bit unsigned integers. Port numbers 0 to 1023 are well-known ports assigned to common services by the *Internet Assigned Numbers Authority (IANA)*. The IANA is a private US nonprofit organization that globally allocates IP addresses and port numbers. For example, HTTP uses port 80. Port 443 is the HTTPS port. SSH servers typically listen on port 22. (These well-known ports are guidelines. An HTTP server may listen to any port, not just port 80.)

Despite these ports being well-known, there is no restriction on which ports services may use. For example, an administrator who wants to obscure a service from attackers expecting it on port 22 may configure an SSH server to listen on port 22422. The IANA designates ports 1024 to 49151 as semi-reserved for lesser common services. Ports 49152 to 65535 are ephemeral ports meant for client socket addresses as recommended by the IANA. (The port range used for client socket addresses is operating-system dependent.)

A common example of port usage is the interaction between your web browser and a web server. Your web browser opens a socket with the operating system, which assigns an address to the socket. Your web browser sends a request through the socket to port 80 on the web server. The web server sends its response to the socket address corresponding to the socket your web browser is monitoring. Your operating system receives the response and passes it onto your web browser through the socket. Your web browser's socket address and the web server's socket address (server IP and port 80) uniquely identify this transaction. This allows your operating system to properly demultiplex the response and pass it along to the right application (that is, your web browser).

Network Address Translation

The four billion IPv4 addresses may seem like a lot until you consider there will be an estimated 24.6 billion Internet of Things (IoT) devices by 2025, according to the Ericsson Mobility Report of June 2020 (*https://www.ericsson.com/en/mobility-report/reports/june-2020/iot-connections-outlook/*). In fact, we've already run out of unreserved IPv4 addresses. The IANA allocated the last IPv4 address block on January 31, 2011.

One way to address the IPv4 shortage is by using *network address translation (NAT)*, a process that allows numerous nodes to share the same public IPv4 address. It requires a device, such as a firewall, load balancer, or router that can keep track of incoming and outgoing traffic and properly route incoming traffic to the correct node.

Figure 2-6 illustrates the NAT process between nodes on a private network and the internet.

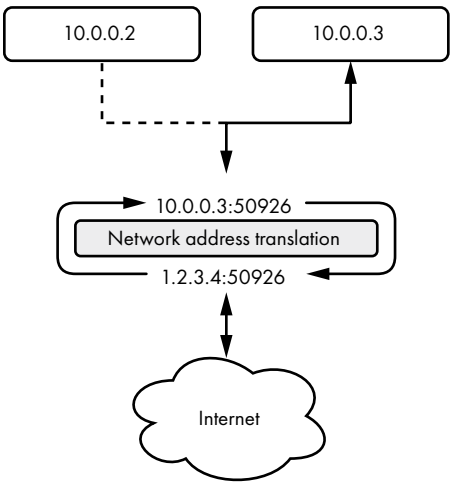

Figure 2-6: Network address translation between a private network and the internet

In Figure 2-6, a NAT-capable device receives a connection from the client socket address 10.0.0.3:50926 destined for a host on the internet. First, the NAT device opens its own connection to the destination host using its public IP 1.2.3.4, preserving the client's socket address port. Its socket address for this transaction is 1.2.3.4:50926. If a client is already using port 50926, the NAT device chooses a random port for its socket address. Then, the NAT device sends the request to the destination host and receives the response on its 1.2.3.4:50926 socket. The NAT device knows which client receives the response because it translates its socket address to the client socket address that established the connection. Finally, the client receives the destination host's response from the NAT device.

The important thing to remember with network address translation is that a node's private IPv4 address behind a NAT device is not visible or directly accessible to other nodes outside the network address–translated

network segment. If you're writing a service that needs to provide a public address for its clients, you may not be able to rely on your node's private IPv4 address if it's behind a NAT device. Hosts outside the NAT device's private network cannot establish incoming connections. Only clients in the private network may establish connections through the NAT device. Instead, your service must rely on the NAT device's properly forwarding a port from its public IP to a socket address on your node.

Unicasting, Multicasting, and Broadcasting

Sending packets from one IP address to another IP address is known as *unicast addressing*. But TCP/IP's internet layer supports IP *multicasting*, or sending a single message to a group of nodes. You can think of it as an opt-in mailing list, such as a newspaper subscription.

From a network programming perspective, multicasting is simple. Routers and switches typically replicate the message for us, as shown in Figure 2-7. We'll discuss multicasting later in this book.

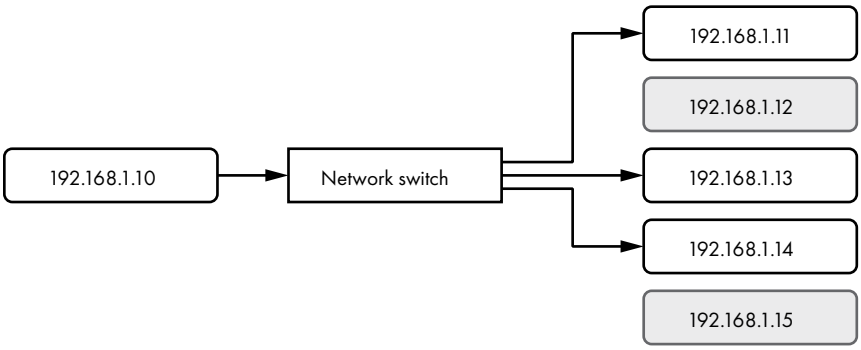

Figure 2-7: The 192.168.1.10 node sending a packet to a subset of network addresses

Broadcasting is the ability to concurrently deliver a message to all IP addresses in a network. To do this, nodes on a network send packets to the *broadcast address* of a subnet. A network switch or router then propagates the packets out to all IPv4 addresses in the subnet (Figure 2-8).

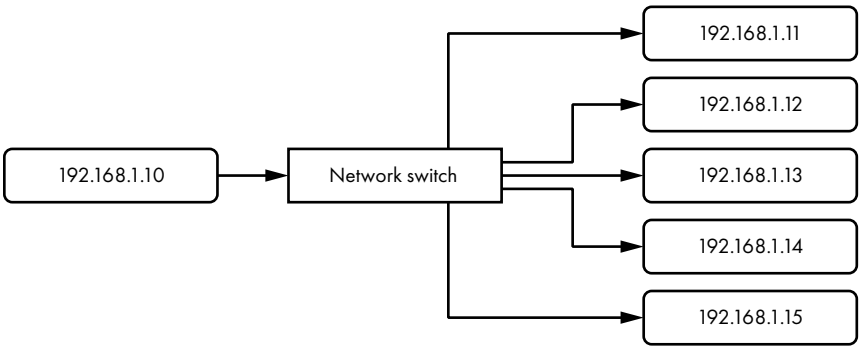

Figure 2-8: The 192.168.1.10 node sending a packet to all addresses on its subnet

Unlike multicasting, the nodes in the subnet don't first need to opt in to receiving broadcast messages. If the node at 192.168.1.10 in Figure 2-8 sends a packet to the broadcast address of its subnet, the network switch will deliver a copy of that packet to the other five IPv4 addresses in the same subnet.

Resolving the MAC Address to a Physical Network Connection

Recall from Chapter 1 that every network interface has a MAC address uniquely identifying the node's physical connection to the network. The MAC address is relevant to only the local network, so routers cannot use a MAC address to route data across network boundaries. Instead, they can route traffic across network boundaries by using an IPv4 address. Once a packet reaches the local network of a destination node, the router sends the data to the destination node's MAC address and, finally, to the destination node's physical network connection.

The *Address Resolution Protocol (ARP)*, detailed in RFC 826 (*https://tools.ietf.org/html/rfc826/*), finds the appropriate MAC address for a given IP address—a process called *resolving* the MAC address. Nodes maintain ARP tables that map an IPv4 address to a MAC address. If a node does not have an entry in its ARP table for a destination IPv4 address, the node will send a request to the local network's broadcast address asking, "Who has this IPv4 address? Please send me your MAC address. Oh, and here is my MAC address." The destination node will receive the ARP request and respond with an ARP reply to the originating node. The originating node will then send the data to the destination node's MAC address. Nodes on the network privy to this conversation will typically update their ARP tables with the values.

IPv6 Addressing

Another solution to the IPv4 shortage is to migrate to the next generation of IP addressing, IPv6. *IPv6 addresses* are 128-bit numbers arranged in eight colon-separated groups of 16 bits, or *hextets*. There are more than 340 undecillion (2^{128}) IPv6 addresses.

Writing IPv6 Addresses

In binary form, IPv6 addresses are a bit ridiculous to write. In the interest of legibility and compactness, we write IPv6 addresses with lowercase hexadecimal values instead.

NOTE *IPv6 hexadecimal values are case-insensitive. However, the Internet Engineering Task Force (IETF) recommends using lowercase values.*

A hexadecimal (hex) digit represents 4 bits, or a *nibble*, of an IPv6 address. For example, we'd represent the two nibbles 1111 1111 in their hexadecimal equivalent of `ff`. Figure 2-9 illustrates the same IPv6 address in binary and hex.

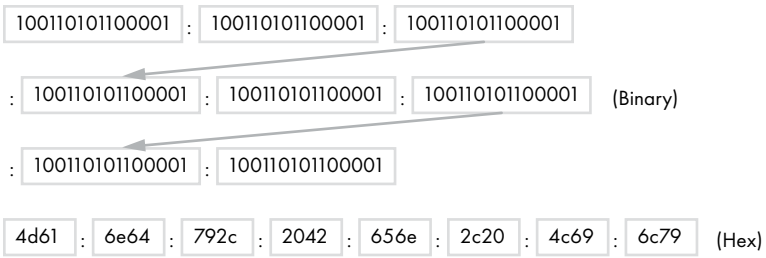

Figure 2-9: Binary and hex representations of the same IPv6 address

Even though hexadecimal IPv6 addresses are a bit more succinct than their binary equivalent, we still have some techniques available to us to simplify them a bit more.

Simplifying IPv6 Addresses

An IPv6 address looks something like this: fd00:4700:0010:0000:0000:0000:6814:d103. That's quite a bit harder to remember than an IPv4 address. Thankfully, you can improve the IPv6 address's presentation to make it more readable by following a few rules.

First, you can remove all leading zeros in each hextet. This simplifies your address without changing its value. It now looks like this: fd00:4700:10:0:0:0:6814:d103. Better, but still long.

Second, you can replace the leftmost group of consecutive, zero-value hextets with double colons, producing the shorter fd00:4700:10::6814:d103. If your address has more than one group of consecutive zero-value hextets, you can remove only the leftmost group. Otherwise, it's impossible for routers to accurately determine the number of hextets to insert when repopulating the full address from its compressed representation. For example, fd00:4700:0000:0000:ef81:0000:6814:d103 rewrites to fd00:4700::ef81:0:6814:d103. The best you could do with the sixth hextet is to remove the leading zeros.

IPv6 Network and Host Addresses

Like IPv4 addresses, IPv6 addresses have a network address and a host address. IPv6's host address is commonly known as the *interface ID*. The network and host addresses are both 64 bits, as shown in Figure 2-10. The first 48 bits of the network address are known as the *global routing prefix (GRP)*, and the last 16 bits of the network address are called the *subnet ID*. The 48-bit GRP is used for globally subdividing the IPv6 address space and routing traffic between these groups. The subnet ID is used to further subdivide each GRP-unique network into site-specific networks. If you run a large ISP, you are assigned one or more GRP-unique blocks of IPv6 addresses. You can then use the subnet ID in each network to further subdivide your allocated IPv6 addresses to your customers.

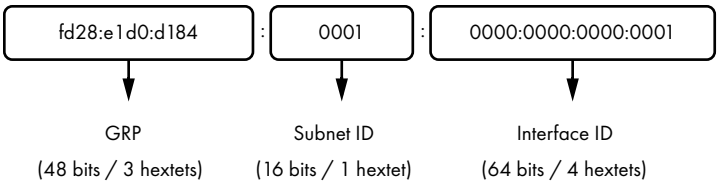

Figure 2-10: IPv6 global routing prefix, subnet ID, and interface ID

The GRP gets determined for you when you request a block of IPv6 addresses from your ISP. IANA assigns the first hextet of the GRP to a regional internet registry (an organization that handles the allocation of addresses for a global region). The regional internet registry then assigns the second GRP hextet to an ISP. The ISP finally assigns the third GRP hextet before assigning a 48-bit subnet of IPv6 addresses to you.

NOTE *For more information on the allocation of IPv6 addresses, see IANA's "IPv6 Global Unicast Address Assignments" document at* https://www.iana.org/assignments/ipv6-unicast-address-assignments/ipv6-unicast-address-assignments.xml.

The first hextet of an IPv6 address gives you a clue to its use. Addresses beginning with the prefix 2000::/3 are meant for global use, meaning every node on the internet will have an IPv6 address starting with 2 or 3 in the first hex. The prefix fc00::/7 designates a unique local address like the 127.0.0.0/8 subnet in IPv4.

NOTE *IANA's "Internet Protocol Version 6 Address Space" document at* https://www.iana.org/assignments/ipv6-address-space/ipv6-address-space.xhtml *provides more details.*

Let's assume your ISP assigned the 2600:fe56:7891::/48 netblock to you. Your 16-bit subnet ID allows you to further subdivide your netblock into a maximum of 65,536 (2^{16}) subnets. Each of those subnets supports over 18 quintillion (2^{64}) hosts. If you assign 1 to the subnet as shown in Figure 2-10, you'd write the full network address as 2600:fe56:7891:1::/64 after removing leading zeros and compressing zero value hextets. Further subnetting your netblock may look like this: 2600:fe56:7891:**2**::/64, 2600:fe56:7891:**3**::/64, 2600:fe56:7891:**4**::/64.

IPv6 Address Categories

IPv6 addresses are divided into three categories: anycast, multicast, and unicast. Notice there is no broadcast type, as in IPv4. As you'll see, anycast and multicast addresses fulfill that role in IPv6.

Unicast Addresses

A *unicast* IPv6 address uniquely identifies a node. If an originating node sends a message to a unicast address, only the node with that address will receive the message, as shown in Figure 2-11.

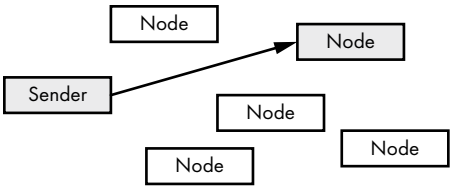

Figure 2-11: Sending to a unicast address

Multicast Addresses

Multicast addresses represent a group of nodes. Whereas IPv4 broadcast addresses will propagate a message out to all addresses on the network, multicast addresses will simultaneously deliver a message to a subset of network addresses, not necessarily all of them, as shown in Figure 2-12.

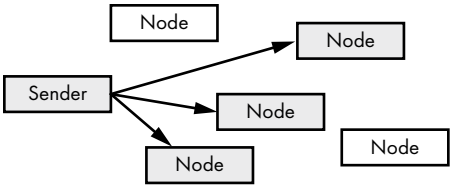

Figure 2-12: Sending to a multicast address

Multicast addresses use the prefix ff00::/8.

Anycast Addresses

Remember that IPv4 addresses must be unique per network segment, or network communication issues can occur. But IPv6 includes support for multiple nodes using the same network address. An *anycast* address represents a group of nodes listening to the same address. A message sent to an anycast address goes to the nearest node listening to the address. Figure 2-13 represents a group of nodes listening to the same address, where the nearest node to the sender receives the message. The sender could transmit to any of the nodes represented by the dotted lines, but sends to the nearest node (solid line).

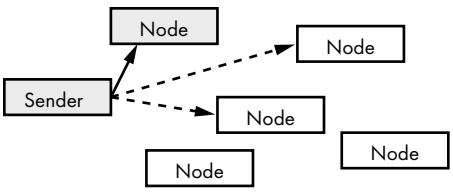

Figure 2-13: Sending to an anycast address

The nearest node isn't always the most physically close node. It is up to the router to determine which node receives the message, usually the node

with the least latency between the origin and the destination. Aside from reducing latency, anycast addressing increases redundancy and can geolocate services.

Sending traffic around the world takes a noticeable amount of time, to the point that the closer you are to a service provider's servers, the better performance you'll experience. Geolocating services across the internet is a common method of placing servers geographically close to users to make sure performance is optimal for all users across the globe. It's unlikely you access servers across an ocean when streaming Netflix. Instead, Netflix geolocates servers close to you so that your experience is ideal.

Advantages of IPv6 over IPv4

Aside from the ridiculously large address space, IPv6 has inherent advantages over IPv4, particularly with regard to efficiency, autoconfiguration, and security.

Simplified Header Format for More Efficient Routing

The IPv6 header is an improvement over the IPv4 header. The IPv4 header contains mandatory yet rarely used fields. IPv6 makes these fields optional. The IPv6 header is extensible, in that functionality can be added without breaking backward compatibility. In addition, the IPv6 header is designed for improved efficiency and reduced complexity over the IPv4 header.

IPv6 also lessens the loads on routers and other hops by ensuring that headers require minimal processing, eliminating the need for checksum calculation at every hop.

Stateless Address Autoconfiguration

Administrators manually assign IPv4 addresses to each node on a network or rely on a service to dynamically assign addresses. Nodes using IPv6 can automatically configure or derive their IPv6 addresses through *stateless address autoconfiguration (SLAAC)* to reduce administrative overhead.

When connected to an IPv6 network, a node can solicit the router for its network address parameters using the *Neighbor Discovery Protocol (NDP)*. NDP leverages the Internet Control Message Protocol, discussed later in this chapter, for router solicitation. It performs the same duties as IPv4's ARP. Once the node receives a reply from the router with the 64-bit network address, the node can derive the 64-bit host portion of its IPv6 address on its own using the 48-bit MAC address assigned to its network interface. The node appends the 16-bit hex FFFE to the first three octets of the MAC address known as the *originally unique identifier (OUI)*. To this, the node appends the remaining three octets of the MAC address, the network interface controller (NIC) identifier. The result is a unique 64-bit interface ID, as shown in Figure 2-14. SLAAC works only in the presence of a router that can respond with router advertisement packets. *Router advertisement packets* contain information clients need to automatically configure their IPv6 address, including the 64-bit network address.

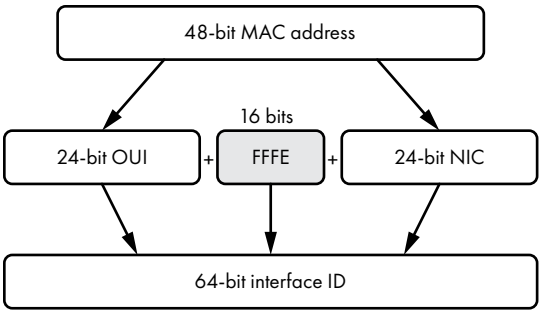

Figure 2-14: Deriving the interface ID from the MAC address

If you value your privacy, the method SLAAC uses to derive a unique interface ID should concern you. No matter which network your device is on, SLAAC will make sure the host portion of your IPv6 address contains your NIC's MAC address. The MAC address is a unique fingerprint that betrays the hardware you use and allows anyone to track your online activity. Thankfully, many people raised these concerns, and SLAAC gained privacy extensions (*https://tools.ietf.org/html/rfc4941/*), which randomize the interface ID. Because of this randomization, it's possible for more than one node on a network to generate the same interface ID. Thankfully, the NDP will automatically detect and fix any duplicate interface ID for you.

Native IPsec Support

IPv6 has native support for *IPsec*, a technology that allows multiple nodes to dynamically create secure connections between each other, ensuring that traffic is encrypted.

NOTE *RFC 6434 made IPsec optional for IPv6 implementations.*

The Internet Control Message Protocol

The Internet Protocol relies on the *Internet Control Message Protocol (ICMP)* to give it feedback about the local network. ICMP can inform you of network problems, unreachable nodes or networks, local network configuration, proper traffic routes, and network time-outs. Both IPv4 and IPv6 have their own ICMP implementations, designated ICMPv4 and ICMPv6, respectively.

Network events often result in ICMP response messages. For instance, if you attempt to send data to an unreachable node, a router will typically respond with an ICMP *destination unreachable* message informing you that your data couldn't reach the destination node. A node may become unreachable if it runs out of resources and can no longer respond to incoming data or if data cannot route to the node. Disconnecting a node from a network will immediately make it unreachable.

Routers use ICMP to help inform you of better routes to your destination node. If you send data to a router that isn't the appropriate or best

router to handle traffic for your destination, it may reply with an ICMP *redirect* message after forwarding your data onto the correct router. The ICMP redirect message is the router's way of telling you to send your data to the appropriate router in the future.

You can determine whether a node is online and reachable by using an ICMP *echo* request (also called a *ping*). If the destination is reachable and receives your ping, it will reply with its own ICMP *echo reply* message (also called a *pong*). If the destination isn't reachable, the router will respond with a destination unreachable message.

ICMP can also notify you when data reaches the end of its life before delivery. Every IP packet has a *time-to-live* value that dictates the maximum number of hops the packet can take before its lifetime expires. The packet's time-to-live value is a counter and decrements by one for every hop it takes. You will receive an ICMP *time exceeded* message if the packet you sent doesn't reach its destination by the time its time-to-live value reaches zero.

IPv6's NDP relies heavily on ICMP router solicitation messages to properly configure a node's NIC.

Internet Traffic Routing

Now that you know a bit about internet protocol addressing, let's explore how packets make their way across the internet from one node to another using those addresses. In Chapter 1, we discussed how data travels down the network stack of the originating node, across a physical medium, and up the stack of the destination node. But in most cases, nodes won't have a direct connection, so they'll have to make use of intermediate nodes to transfer data. Figure 2-15 shows that process.

The intermediate nodes (Nodes 1 and 2 in Figure 2-15) are typically routers or firewalls that control the path data takes from one node to the other. *Firewalls* control the flow of traffic in and out of a network, primarily to secure networks behind the firewall.

No matter what type of node they are, intermediate nodes have a network stack associated with each network interface. In Figure 2-15, Node 1 receives data on its incoming network interface. The data climbs the stack to Layer 3, where it's handed off to the outgoing network interface's stack. The data then makes its way to Node 2's incoming network interface before ultimately being routed to the server.

The incoming and outgoing network interfaces in Node 1 and Node 2 may send data over different media types using IPv4, so they must use encapsulation to isolate the implementation details of each media type from the data being sent. Let's assume Node 1 receives data from the client over a wireless network and it sends data to Node 2 over an Ethernet connection. Node 1's incoming Layer 1 knows how to convert the radio signals from the wireless network into bits. Layer 1 sends the bits up to Layer 2. Layer 2 converts the bits to a frame and extracts the packet and sends it up to Layer 3.

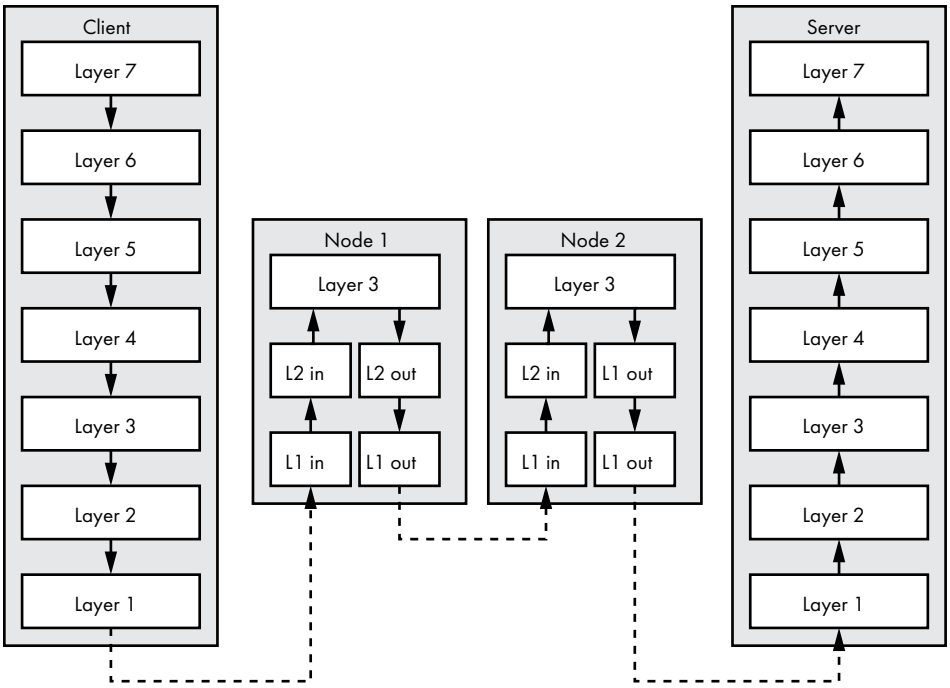

Figure 2-15: Routing packets through two hops

Layer 3 on both the incoming and outgoing NICs speak IPv4, which routes the packet between the two interface network stacks. The outgoing NIC's Layer 2 receives the packet from its Layer 3 and encapsulates it before sending the frame onto its Layer 1 as bits. The outgoing Layer 1 converts the bits into electric signals suitable for transmission over Ethernet. The data in transport from the client's Layer 7 never changed despite the data's traversing multiple nodes over different media on its way to the destination server.

Routing Protocols

The routing overview in Figure 2-15 makes the process look easy, but the routing process relies on a symphony of protocols to make sure each packet reaches its destination no matter the physical medium traversed or network outages along the way. Routing protocols have their own criteria for determining the best path between nodes. Some protocols determine a route's efficiency based on hop count. Some may use bandwidth. Others may use more complicated means to determine which route is the most efficient.

Routing protocols are either internal or external depending on whether they route packets within an autonomous system or outside of one. An *autonomous system* is an organization that manages one or more networks. An ISP is an example of an autonomous system. Each autonomous system is assigned an autonomous system number (ASN), as outlined in RFC 1930 (*https://tools.ietf.org/html/rfc1930/*). This ASN is used to broadcast an ISP's network

information to other autonomous systems using an external routing protocol. An *external routing protocol* routes data between autonomous systems. The only routing protocol we'll cover is BGP since it is the glue of the internet, binding all ASN-assigned ISPs together. You don't need to understand BGP in depth, but being familiar with it can help you better debug network issues related to your code and improve your code's resiliency.

The Border Gateway Protocol

The *Border Gateway Protocol (BGP)* allows ASN-assigned ISPs to exchange routing information. BGP relies on trust between ISPs. That is, if an ISP says it manages a specific network and all traffic destined for that network should be sent to it, the other ISPs trust this claim and send traffic accordingly. As a result, BGP misconfigurations, or *route leaks*, often result in very public network outages.

In 2008, Pakistan Telecommunications Company effectively took down YouTube worldwide after the Pakistani Ministry of Communications demanded the country block *youtube.com* in protest of a YouTube video. Pakistan Telecom used BGP to send all requests destined for YouTube to a null route, a route that drops all data without notification to the sender. But Pakistan Telecom accidentally leaked its BGP route to the world instead of restricting it to the country. Other ISPs trusted the update and null routed YouTube requests from their clients, making *youtube.com* inaccessible for two hours all over the world.

In 2012, Google's services were rerouted through Indonesia for 27 minutes when the ISP Moratel shared a BGP route directing all Google traffic to Moratel's network as if Moratel was now hosting Google's network infrastructure. There was speculation at the time that the route leakage was malicious, but Moratel blamed a hardware failure.

BGP usually makes news only when something goes wrong. Other times, it plays the silent hero, serving a significant role in mitigating distributed denial-of-service (DDOS) attacks. In a *DDOS attack*, a malicious actor directs traffic from thousands of compromised nodes to a victim node with the aim of overwhelming the victim and consuming all its bandwidth, effectively denying service to legitimate clients. Companies that specialize in mitigating DDOS attacks use BGP to reroute all traffic destined for the victim node to their AS networks, filter out the malicious traffic from the legitimate traffic, and route the sanitized traffic back to the victim, nullifying the effects of the attack.

Name and Address Resolution

The *Domain Name System (DNS)* is a way of matching IP addresses to *domain names*, which are the names we enter in an address bar when we want to visit websites. Although the internet protocol uses IP addresses to locate hosts, domain names (like *google.com*) are easier for humans to understand and remember. If I gave you the IP address 172.217.6.14 to visit, you wouldn't know who owned that IP address or what I was directing you to visit. But if

I gave you *google.com* instead, you'd know exactly where I was sending you. DNS allows you to remember a hostname instead of its IP address in the same way that your smartphone's contact list frees you from having to memorize all those phone numbers.

All domains are children of a *top-level domain*, such as *.com*, *.net*, *.org*, and so on. Take *nostarch.com*, for instance. No Starch Press registered the *nostarch* domain on the *.com* top-level domain from a registrar with the authority from IANA to register *.com* domains. No Starch Press now has the exclusive authority to manage DNS records for *nostarch.com* and publish records on its DNS server. This includes the ability for No Starch Press to publish *subdomains*—a subdivision of a domain—under its domain. For example, *maps.google.com* is a subdomain of *google.com*. A longer example is *sub3.sub2.sub1.domain.com*, where *sub3* is a subdomain under *sub2.sub1.domain.com*, *sub2* is subdomain under *sub1.domain.com*, and *sub1* is a subdomain under *domain.com*.

If you enter `https://nostarch.com` in your web browser, your computer will consult its configured *domain name resolver*, a server that knows how to retrieve the answer to your query. The resolver will start by asking one of the 13 IANA-maintained root name servers for the IP address of *nostarch.com*. The root name server will examine the top-level domain of the domain you requested and give your resolver the address of the *.com* name server. Your resolver will then ask the *.com* name server for *nostarch.com*'s IP address, which will examine the domain portion and direct your resolver to ask No Starch Press's name server. Finally, your resolver will ask No Starch Press's name server and receive the IP address that corresponds to *nostarch.com*. Your web browser will establish a connection to this IP address, retrieve the web page, and render it for you. This hierarchical journey of domain resolution allows you to zero in on a specific web server, and all you had to know was the domain name. No Starch Press is free to move its servers to a different ISP with new IP addresses, and yet you'll still be able to visit its website by using *nostarch.com*.

Domain Name Resource Records

Domain name servers maintain *resource records* for the domains they serve. Resource records contain domain-specific information, used to satisfy domain name queries, like IP addresses, mail server hostnames, mail-handling rules, and authentication tokens. There are many resource records, but this section focuses on only the most common ones: address records, start-of-authority records, name server records, canonical name records, mail exchange records, pointer records, and text records.

NOTE *For more details on types of resource records, see Wikipedia's entry at* https://en.wikipedia.org/wiki/List_of_DNS_record_types.

Our exploration of each resource record will use a utility called *dig* to query domain name servers. This utility may be available on your operating system, but in case you don't have dig installed, you can use the G Suite Toolbox Dig utility (*https://toolbox.googleapps.com/apps/dig/*) in a web browser

and receive similar output. All domain names you'll see are *fully qualified*, which means they end in a period, displaying the domain's entire hierarchy from the root zone. The *root zone* is the top DNS namespace.

Dig's default output includes a bit of data relevant to your query but irrelevant to your study of its output. Therefore, I've elected to snip out header and footer information in dig's output in each example to follow. Also please be aware that the specific output in this book is a snapshot from when I executed each query. It may look different when you execute these commands.

The Address Record

The *Address (A) record* is the most common record you'll query. An A record will resolve to one or more IPv4 addresses. When your computer asks its resolver to retrieve the IP address for *nostarch.com*, the resolver ultimately asks the domain name server for the *nostarch.com* Address (A) resource record. Listing 2-1 shows the question and answer sections when you query for the *google.com* A record.

```
$ dig google.com. a
-- snip --
❶ ;QUESTION
❷ google.com. ❸IN ❹A
❺ ;ANSWER
❻ google.com. ❼299 IN A ❽172.217.4.46
-- snip --
```

Listing 2-1: DNS answer of the google.com *A resource record*

Each section in a DNS reply begins with a header ❶, prefixed with a semicolon to indicate that the line is a comment rather than code to be processed. Within the question section, you ask the domain name server for the domain name *google.com* ❷ with the class `IN` ❸, which indicates that this record is internet related. You also use `A` to ask for the A record ❹ specifically.

In the Answer section ❺, the domain name server resolves the *google.com* A record to six IPv4 addresses. The first field of each returned line is the domain name ❻ you queried. The second field is the TTL value ❼ for the record. The *TTL value* tells domain name resolvers how long to cache or remember the record, and it lets you know how long you have until the cached record expires. When you request a DNS record, the domain name resolver will first check its cache. If the answer is in its cache, it will simply return the cached answer instead of asking the domain name server for the answer. This improves domain name resolution performance for records that are unlikely to change frequently. In this example, the record will expire in 299 seconds. The last field is the IPv4 address ❽. Your web browser could use any one of the six IPv4 addresses to establish a connection to *google.com*.

The AAAA resource record is the IPv6 equivalent of the A record.

The Start of Authority Record

The *Start of Authority (SOA) record* contains authoritative and administrative details about the domain, as shown in Listing 2-2. All domains must have an SOA record.

```
$ dig google.com. soa
-- snip --
;QUESTION
google.com. IN SOA
;ANSWER
google.com. 59 IN SOA ❶ns1.google.com. ❷dns-admin.google.com. ❸248440550
900 900 1800 60
-- snip --
```

Listing 2-2: DNS answer of the google.com *SOA resource record*

The first four fields of an SOA record are the same as those found in an A record. The SOA record also includes the primary name server ❶, the administrator's email address ❷, and fields ❸ used by secondary name servers outside the scope of this book. Domain name servers primarily consume SOA records. However, the email address is useful if you wish to contact the domain's administrator.

NOTE *The administrator's email address is encoded as a name, with the at sign (@) replaced by a period.*

The Name Server Record

The *Name Server (NS) record* returns the authoritative name servers for the domain name. *Authoritative name servers* are the name servers able to provide answers for the domain name. NS records will include the primary name server from the SOA record and any secondary name servers answering DNS queries for the domain. Listing 2-3 is an example of the NS records for *google.com*.

```
$ dig google.com. ns
-- snip --
;QUESTION
google.com. IN NS
;ANSWER
google.com. 21599 IN NS ❶ns1.google.com.
google.com. 21599 IN NS ns2.google.com.
google.com. 21599 IN NS ns3.google.com.
google.com. 21599 IN NS ns4.google.com.
-- snip --
```

Listing 2-3: DNS answer of the google.com *NS resource records*

Like the CNAME record, discussed next, the NS record will return a fully qualified domain name ❶, not an IP address.

The Canonical Name Record

The *Canonical Name (CNAME) record* points one domain at another. Listing 2-4 shows a CNAME record response. CNAME records can make administration a bit easier. For example, you can create one named *mail.yourdomain.com* and direct it to Gmail's login page. This not only is easier for your users to remember but also gives you the flexibility of pointing the CNAME at another email provider in the future without having to inform your users.

```
$ dig mail.google.com. a
-- snip --
;QUESTION
mail.google.com. IN A
;ANSWER
❶ mail.google.com. 21599 IN CNAME ❷googlemail.l.google.com.
googlemail.l.google.com. 299 IN A 172.217.3.229
-- snip --
```

Listing 2-4: DNS answer of the mail.google.com *CNAME resource record*

Notice that you ask the domain name server for the A record of the subdomain *mail.google.com*. But in this case, you receive a CNAME instead. This tells you that *googlemail.l.google.com* ❷ is the canonical name for *mail.google.com* ❶. Thankfully, you receive the A record for *googlemail.l.google.com* with the response, alleviating you from having to make a second query. You now know your destination IP address is 172.217.3.229. Google's domain name server was able to return both the CNAME answer and the corresponding Address answer in the same reply because it is an authority for the CNAME answer's domain name as well. Otherwise, you would expect only the CNAME answer and would then need to make a second query to resolve the CNAME answer's IP address.

The Mail Exchange Record

The *Mail Exchange (MX) record* specifies the mail server hostnames that should be contacted when sending email to recipients at the domain. Remote mail servers will query the MX records for the domain portion of a recipient's email address to determine which servers should receive mail for the recipient. Listing 2-5 shows the response a mail server will receive.

```
$ dig google.com. mx
-- snip --
;QUESTION
google.com. IN MX
;ANSWER
google.com. 599 IN MX ❶10 aspmx.l.google.com.
google.com. 599 IN MX 50 alt4.aspmx.l.google.com.
google.com. 599 IN MX 30 alt2.aspmx.l.google.com.
google.com. 599 IN MX 20 alt1.aspmx.l.google.com.
google.com. 599 IN MX 40 alt3.aspmx.l.google.com.
-- snip --
```

Listing 2-5: DNS answer of the google.com *MX resource records*

In addition to the domain name, TTL value, and record type, MX records contain the *priority field* ❶, which rates the priority of each mail server. The lower the number, the higher the priority of the mail server. Mail servers attempt to deliver emails to the mail server with the highest priority, then resort to the mail servers with the next highest priority if necessary. If more than one mail server shares the same priority, the mail server will pick one at random.

The Pointer Record

The *Pointer (PTR) record* allows you to perform a reverse lookup by accepting an IP address and returning its corresponding domain name. Listing 2-6 shows the reverse lookup for 8.8.4.4.

```
$ dig 4.4.8.8.in-addr.arpa. ptr
-- snip --
;QUESTION
❶ 4.4.8.8.in-addr.arpa. IN PTR
;ANSWER
4.4.8.8.in-addr.arpa. 21599 IN PTR ❷google-public-dns-b.google.com.
-- snip --
```

Listing 2-6: DNS answer of the 8.8.4.4 PTR resource record

To perform the query, you ask the domain name server for the IPv4 address in reverse order ❶ with the special domain *in-addr.arpa* appended because the reverse DNS records are all under the *.arpa* top-level domain. For example, querying the pointer record for the IP 1.2.3.4 means you need to ask for *4.3.2.1.in-addr.arpa*. The query in Listing 2-6 tells you that the IPv4 address 8.8.4.4 reverses to the domain name *google-public-dns-b.google.com* ❷. If you were performing a reverse lookup of an IPv6 address, you'd append the special domain *ip6.arpa* to the reversed IPv6 address as you did for the IPv4 address.

NOTE *See Wikipedia for more information on reverse DNS lookup:* https://en.wikipedia.org/wiki/Reverse_DNS_lookup.

The Text Record

The *Text (TXT) record* allows the domain owner to return arbitrary text. These records can contain values that prove domain ownership, values that remote mail servers can use to authorize email, and entries to specify which IP addresses may send mail on behalf of the domain, among other uses. Listing 2-7 shows the text records associated with *google.com*.

```
$ dig google.com. txt
-- snip --
;QUESTION
google.com. IN TXT
;ANSWER
```

```
google.com. 299 IN TXT
  ❶"facebook-domain-verification=22rm551cu4k0ab0bxsw536tlds4h95"
google.com. 299 IN TXT "docusign=05958488-4752-4ef2-95eb-aa7ba8a3bd0e"
google.com. 299 IN TXT ❷"v=spf1 include:_spf.google.com ~all"
google.com. 299 IN TXT
    "globalsign-smime-dv=CDYX+XFHUw2wml6/Gb8+59BsH31KzUr6c1l2BPvqKX8="
-- snip --
```

Listing 2-7: DNS answer of the google.com TXT resource records

The domain queries and answers should start to look familiar by now. The last field in a TXT record is a string of the TXT record value ❶. In this example, the field has a Facebook verification key, which proves to Facebook that Google's corporate Facebook account is who they say they are and has the authority to make changes to Google's content on Facebook. It also contains *Sender Policy Framework* rules ❷, which inform remote mail servers which IP addresses may deliver email on Google's behalf.

NOTE *The Facebook for Developers site has more information about domain verification at* https://developers.facebook.com/docs/sharing/domain-verification/.

Multicast DNS

Multicast DNS (mDNS) is a protocol that facilitates name resolution over a local area network (LAN) in the absence of a DNS server. When a node wants to resolve a domain name to an IP address, it will send a request to an IP multicast group. Nodes listening to the group receive the query, and the node with the requested domain name responds to the IP multicast group with its IP address. You may have used mDNS the last time you searched for and configured a network printer on your computer.

Privacy and Security Considerations of DNS Queries

DNS traffic is typically unencrypted when it traverses the internet. A potential exception occurs if you're connected to a virtual private network (VPN) and are careful to make sure all DNS traffic passes through its encrypted tunnel. Because of DNS's unencrypted transport, unscrupulous ISPs or intermediate providers may glean sensitive information in your DNS queries and share those details with third parties. You can make a point of visiting HTTPS-only websites, but your DNS queries may betray your otherwise secure browsing habits and allow the DNS server's administrators to glean the sites you visit.

Security is also a concern with plaintext DNS traffic. An attacker could convince your web browser to visit a malicious website by inserting a response to your DNS query. Considering the difficulty of pulling off such an attack, it's not an attack you're likely to experience, but it's concerning nonetheless. Since DNS servers often cache responses, this attack usually takes place between your device and the DNS server it's configured to use. RFC 7626 (*https://tools.ietf.org/html/rfc7626/*) covers these topics in more detail.

Domain Name System Security Extensions

Generally, you can ensure the authenticity of data sent over a network in two ways: authenticating the content and authenticating the channel. *Domain Name System Security Extensions (DNSSEC)* is a method to prevent the covert modification of DNS responses in transit by using digital signatures to authenticate the response. DNSSEC ensures the authenticity of data by authenticating the content. DNS servers cryptographically sign the resource records they serve and make those signatures available to you. You can then validate the responses from authoritative DNS servers against the signatures to make sure the responses aren't fraudulent.

DNSSEC doesn't address privacy concerns. DNSSEC queries still traverse the network unencrypted, allowing for passive observation.

DNS over TLS

DNS over TLS (DoT), detailed in RFC 7858 (*https://tools.ietf.org/html/rfc7858/*), addresses both security and privacy concerns by using *Transport Layer Security (TLS)* to establish an encrypted connection between the client and its DNS server. TLS is a common protocol used to provide cryptographically secure communication between nodes on a network. Using TLS, DNS requests and responses are fully encrypted in transit, making it impossible for an attacker to eavesdrop on or manipulate responses. DoT ensures the authenticity of data by authenticating the channel. It does not need to rely on cryptographic signatures like DNSSEC because the entire conversation between the DNS server and the client is encrypted.

DoT uses a different network port than does regular DNS traffic.

DNS over HTTPS

DNS over HTTPS (DoH), detailed in RFC 8484 (*https://tools.ietf.org/html/rfc8484/*) aims to address DNS security and privacy concerns while using a heavily used TCP port. Like DoT, DoH sends data over an encrypted connection, authenticating the channel. DoH uses a common port and maps DNS requests and responses to HTTP requests and responses. Queries over HTTP can take advantage of all HTTP features, such as caching, compression, proxying, and redirection.

What You've Learned

We covered a lot of ground in this chapter. You learned about IP addressing, starting with the basics of IPv4 multicasting, broadcasting, TCP and UDP ports, socket addresses, network address translation, and ARP. You then learned about IPv6, its address categories, and its advantages over IPv4.

You learned about the major network-routing protocols, ICMP and DNS. I'll again recommend the *TCP/IP Guide* by Charles M. Kozierok (No Starch Press, 2005) for its extensive coverage of the topics in this chapter.

PART II

SOCKET-LEVEL PROGRAMMING

3

RELIABLE TCP DATA STREAMS

TCP allows you to reliably stream data between nodes on a network. This chapter takes a deeper dive into the protocol, focusing on the aspects directly influenced by the code we'll write to establish TCP connections and transmit data over those connections. This knowledge should help you debug network-related issues in your programs.

We'll start by covering the TCP handshake process, its sequence numbers, acknowledgments, retransmissions, and other features. Next, we'll implement the steps of a TCP session in Go, from dialing, listening, and accepting to the session termination. Then, we'll discuss time-outs and temporary errors, how to detect them, and how to use them to keep our users happy. Finally, we'll cover the early detection of unreliable network connections. Go's standard library allows you to write robust TCP-based networking applications. But it doesn't hold your hand. If you aren't mindful of managing incoming data or properly closing connections, you'll experience insidious bugs in your programs.

What Makes TCP Reliable?

TCP is reliable because it overcomes the effects of packet loss or receiving packets out of order. *Packet loss* occurs when data fails to reach its destination—typically because of data transmission errors (such as wireless network interference) or network congestion. *Network congestion* happens when nodes attempt to send more data over a network connection than the connection can handle, causing the nodes to discard the excess packets. For example, you can't send data at a rate of 1 gigabit per second (Gbps) over a 10 megabit-per-second (Mbps) connection. The 10Mbps connection quickly becomes saturated, and nodes involved in the flow of the data drop the excess data.

TCP adapts its data transfer rate to make sure it transmits data as fast as possible while keeping dropped packets to a minimum, even if the network conditions change—for example, the Wi-Fi signal fades, or the destination node becomes overwhelmed with data. This process, called *flow control*, does its best to make up for the deficiencies of the underlying network media. TCP cannot send good data over a bad network and is at the mercy of the network hardware.

TCP also keeps track of received packets and retransmits unacknowledged packets, as necessary. Recipients can also receive packets out of sequence if, for example, data is rerouted in transit. Remember from Chapter 2 that routing protocols use metrics to determine how to route packets. These metrics may change as network conditions change. There is no guarantee that all packets you send take the same route for the duration of the TCP session. Thankfully, TCP organizes unordered packets and processes them in sequence.

Together with flow control and retransmission, these properties allow TCP to overcome packet loss and facilitate the delivery of data to the recipient. As a result, TCP eliminates the need for you to concern yourself with these errors. You are free to focus on the data you send and receive.

Working with TCP Sessions

A *TCP session* allows you to deliver a stream of data of any size to a recipient and receive confirmation that the recipient received the data. This saves you from the inefficiency of sending a large amount of data across a network, only to find out at the end of the transmission that the recipient didn't receive it.

Much like the occasional head nod that people use to indicate they're listening to someone speaking, streaming allows you to receive feedback from the recipient while the transfer is taking place so that you can correct any errors in real time. In fact, you can think of a TCP session as you would a conversation between two nodes. It starts with a greeting, progresses into the conversation, and concludes with a farewell.

As we discuss the specifics of TCP, I want you to understand that Go takes care of the implementation details for you. Your code will take advantage of the `net` package's interfaces when working with TCP connections.

Establishing a Session with the TCP Handshake

A TCP connection uses a three-way handshake to introduce the client to the server and the server to the client. The handshake creates an established TCP session over which the client and server exchange data. Figure 3-1 illustrates the three messages sent in the handshake process.

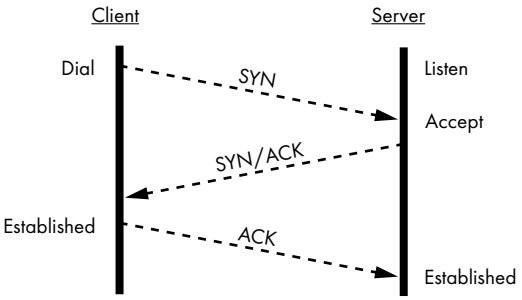

Figure 3-1: The three-way handshake process leading to an established TCP session

Before it can establish a TCP session, the server must listen for incoming connections. (I use the terms *server* and *client* in this chapter to refer to the listening node and dialing node, respectively. TCP itself doesn't have a concept of a client and server, but an established session between two nodes, whereby one node reaches out to another node to establish the session.)

As the first step of the handshake, the client sends a packet with the *synchronize (SYN) flag* to the server. This SYN packet informs the server of the client's capabilities and preferred window settings for the rest of the conversation. We'll discuss the receive window shortly. Next, the server responds with its own packet, with both the *acknowledgment (ACK)* and SYN flags set. The ACK flag tells the client that the server acknowledges receipt of the client's SYN packet. The server's SYN packet tells the client what settings it's agreed to for the duration of the conversation. Finally, the client replies with an ACK packet to acknowledge the server's SYN packet, completing the three-way handshake.

Completion of the three-way handshake process establishes the TCP session, and nodes may then exchange data. The TCP session remains idle until either side has data to transmit. Unmanaged and lengthy idle TCP sessions may result in wasteful consumption of memory. We'll cover techniques for managing idle connections in your code later in this chapter.

When you initiate a connection in your code, Go will return either a connection object or an error. If you receive a connection object, the TCP handshake succeeded. You do not need to manage the handshake yourself.

Acknowledging Receipt of Packets by Using Their Sequence Numbers

Each TCP packet contains a *sequence number*, which the receiver uses to acknowledge receipt of each packet and properly order the packets for presentation to your Go application (Figure 3-2).

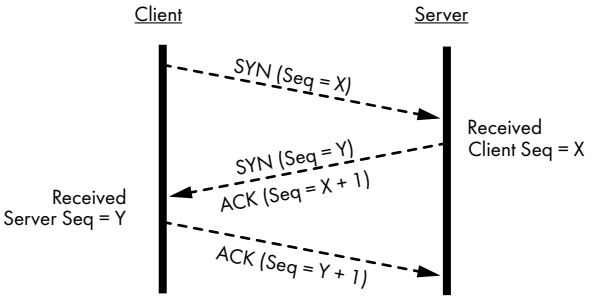

Figure 3-2: Client and server exchanging sequence numbers

The client's operating system determines the initial sequence number (X in Figure 3-2) and sends it to the server in the client's SYN packet during the handshake. The server acknowledges receipt of the packet by including this sequence number in its ACK packet to the client. Likewise, the server shares its generated sequence number Y in its SYN packet to the client. The client replies with its ACK to the server.

An ACK packet uses the sequence number to tell the sender, "I've received all packets up to and including the packet with this sequence number." One ACK packet can acknowledge the receipt of one or more packets from the sender. The sender uses the sequence number in the ACK packet to determine whether it needs to retransmit any packets. For example, if a sender transmits a bunch of packets with sequence numbers up through 100 but then receives an ACK from the receiver with sequence number 90, the sender knows it needs to retransmit packets from sequence numbers 91 to 100.

While writing and debugging network programs, it's often necessary to view the traffic your code sends and receives. To capture and inspect TCP packets, I strongly recommend you familiarize yourself with Wireshark (*https://www.wireshark.org/*). This program will go a long way toward helping you understand how your code influences the data sent over the network. To learn more, see *Practical Packet Analysis*, 3rd Edition, by Chris Sanders (No Starch, 2017).

If you view your application's network traffic in Wireshark, you may notice *selective acknowledgments (SACKs)*. These are ACK packets used to acknowledge the receipt of a *subset* of sent packets. For example, let's assume the sender transmitted a hundred packets but only packets 1 to 59 and 81 to 100 made it to the receiver. The receiver could send a SACK to inform the sender what subset of packets it received.

Here again, Go handles the low-level details. Your code will not need to concern itself with sequence numbers and acknowledgments.

Receive Buffers and Window Sizes

Since TCP allows a single ACK packet to acknowledge the receipt of more than one incoming packet, the receiver must advertise to the sender how much space it has available in its receive buffer before it sends an acknowledgment. A *receive buffer* is a block of memory reserved for incoming data

on a network connection. The receive buffer allows the node to accept a certain amount of data from the network without requiring an application to immediately read the data. Both the client and the server maintain their own per-connection receive buffer. When your Go code reads data from a network connection object, it reads the data from the connection's receive buffer.

ACK packets include a particularly important piece of information: the *window size*, which is the number of bytes the sender can transmit to the receiver without requiring an acknowledgment. If the client sends an ACK packet to the server with a window size of 24,537, the server knows it can send 24,537 bytes to the client before expecting the client to send another ACK packet. A window size of zero indicates that the receiver's buffer is full and can no longer receive additional data. We'll discuss this scenario a bit later in this chapter.

Both the client and the server keep track of each other's window size and do their best to completely fill each other's receive buffers. This method—of receiving the window size in an ACK packet, sending data, receiving an updated window size in the next ACK, and then sending more data—is known as a *sliding window*, as shown in Figure 3-3. Each side of the connection offers up a window of data that can it can receive at any one time.

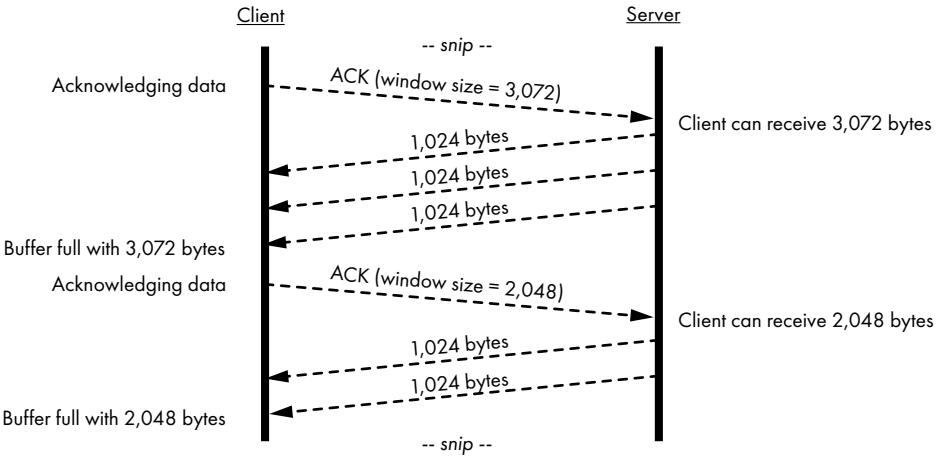

Figure 3-3: A client's ACKs advertising the amount of data it can receive

In this snippet of communication, the client sends an ACK for previously received data. This ACK includes a window size of 3,072 bytes. The server now knows that it can send up to 3,072 bytes before it requires an ACK from the client. The server sends three packets with 1,024 bytes each to fill the client's receive buffer. The client then sends another ACK with an updated window size of 2,048 bytes. This means that the application running on the client read 2,048 bytes from the receive buffer before the client sent its acknowledgment to the server. The server then sends two more packets of 1,024 bytes to fill the client's receive buffer and waits for another ACK.

Here again, all you need to concern yourself with is reading and writing to the connection object Go gives you when you establish a TCP connection. If something goes wrong, Go will surely let you know by returning an error.

Gracefully Terminating TCP Sessions

Like the handshake process, gracefully terminating a TCP session involves exchanging a sequence of packets. Either side of the connection may initiate the termination sequence by sending a *finish (FIN)* packet. In Figure 3-4, the client initiates the termination by sending a FIN packet to the server.

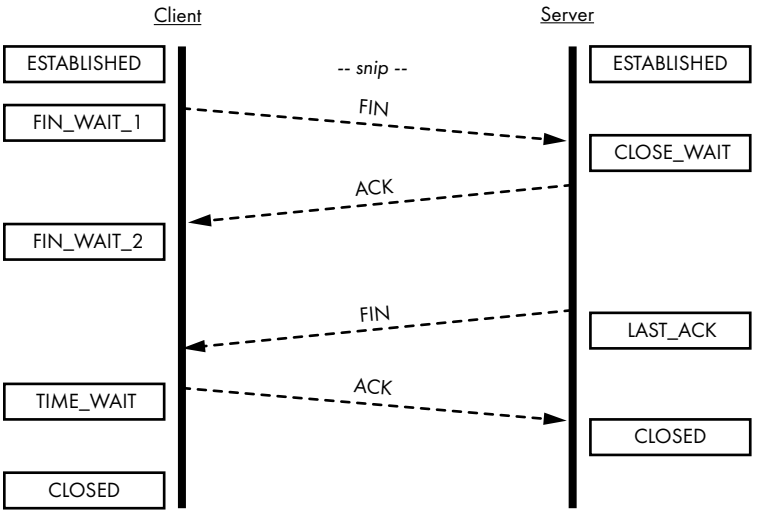

Figure 3-4: The client initiates a TCP session termination with the server.

The client's connection state changes from ESTABLISHED to FIN_WAIT_1, which indicates the client is in the process of tearing down the connection from its end and is waiting for the server's acknowledgment. The server acknowledges the client's FIN and changes its connection state from ESTABLISHED to CLOSE_WAIT. The server sends its own FIN packet, changing its state to LAST_ACK, indicating it's waiting for a final acknowledgment from the client. The client acknowledges the server's FIN and enters a TIME_WAIT state, whose purpose is to allow the client's final ACK packet to reach the server. The client waits for twice the maximum segment lifetime (the segment lifetime arbitrarily defaults to two minutes, per RFC 793, but your operating system may allow you to tweak this value), then changes its connection state to CLOSED without any further input required from the server. The *maximum segment lifetime* is the duration a TCP segment can remain in transit before the sender considers it abandoned. Upon receiving the client's last ACK packet, the server immediately changes its connection state to CLOSED, fully terminating the TCP session.

Like the initial handshake, Go handles the details of the TCP connection teardown process when you close a connection object.

Handling Less Graceful Terminations

Not all connections politely terminate. In some cases, the application that opened a TCP connection may crash or abruptly stop running for some reason. When this happens, the TCP connection is immediately closed. Any packets sent from the other side of the former connection will prompt the closed side of the connection to return a *reset (RST) packet*. The RST packet informs the sender that the receiver's side of the connection closed and will no longer accept data. The sender should close its side of the connection knowing the receiver ignored any packets it did not acknowledge.

Intermediate nodes, such as firewalls, can send RST packets to each node in a connection, effectively terminating the socket from the middle.

Establishing a TCP Connection by Using Go's Standard Library

The `net` package in Go's standard library includes good support for creating TCP-based servers and clients capable of connecting to those servers. Even so, it's your responsibility to make sure you handle the connection appropriately. Your software should be attentive to incoming data and always strive to gracefully shut down the connection. Let's write a TCP server that can listen for incoming TCP connections, initiate connections from a client, accept and asynchronously handle each connection, exchange data, and terminate the connection.

Binding, Listening for, and Accepting Connections

To create a TCP server capable of listening for incoming connections (called a *listener*), use the `net.Listen` function. This function will return an object that implements the `net.Listener` interface. Listing 3-1 shows the creation of a listener.

```
package ch03

import (
    "net"
    "testing"
)

func TestListener(t *testing.T) {
  ❶listener, err := net.Listen("❷tcp", "❸127.0.0.1:0")
    if err != nil {
        t.Fatal(err)
    }
❹   defer func() { _ = listener.Close() }()

    t.Logf("bound to %q", ❺listener.Addr())
}
```

*Listing 3-1: Creating a listener on 127.0.0.1 using a random port (*listen_test.go*)*

The `net.Listen` function accepts a network type ❷ and an IP address and port separated by a colon ❸. The function returns a `net.Listener` interface ❶ and an `error` interface. If the function returns successfully, the listener is bound to the specified IP address and port. *Binding* means that the operating system has exclusively assigned the port on the given IP address to the listener. The operating system allows no other processes to listen for incoming traffic on bound ports. If you attempt to bind a listener to a currently bound port, `net.Listen` will return an error.

You can choose to leave the IP address and port parameters empty. If the port is zero or empty, Go will randomly assign a port number to your listener. You can retrieve the listener's address by calling its `Addr` method ❺. Likewise, if you omit the IP address, your listener will be bound to all unicast and anycast IP addresses on the system. Omitting both the IP address and port, or passing in a colon for the second argument to `net.Listen`, will cause your listener to bind to all unicast and anycast IP addresses using a random port.

In most cases, you should use `tcp` as the network type for `net.Listener`'s first argument. You can restrict the listener to just IPv4 addresses by passing in `tcp4` or exclusively bind to IPv6 addresses by passing in `tcp6`.

You should always be diligent about closing your listener gracefully by calling its `Close` method ❹, often in a `defer` if it makes sense for your code. Granted, this is a test case, and Go will tear down the listener when the test completes, but it's good practice nonetheless. Failure to close the listener may lead to memory leaks or deadlocks in your code, because calls to the listener's `Accept` method may block indefinitely. Closing the listener immediately unblocks calls to the `Accept` method.

Listing 3-2 demonstrates how a listener can accept incoming TCP connections.

```
❶ for {
    ❷conn, err := ❸listener.Accept()
      if err != nil {
          return err
      }

  ❹ go func(c net.Conn) {
  ❺     defer c.Close()

          // Your code would handle the connection here.
      }(conn)
  }
```

Listing 3-2: Accepting and handling incoming TCP connection requests

Unless you want to accept only a single incoming connection, you need to use a `for` loop ❶ so your server will accept each incoming connection, handle it in a goroutine, and loop back around, ready to accept the next connection. Serially accepting connections is perfectly acceptable and efficient, but beyond that point, you should use a goroutine to handle each connection. You could certainly write serialized code after accepting a

connection if your use case demands it, but it would be woefully inefficient and fail to take advantage of Go's strengths. We start the `for` loop by calling the listener's `Accept` method ❷. This method will block until the listener detects an incoming connection and completes the TCP handshake process between the client and the server. The call returns a `net.Conn` interface ❸ and an `error`. If the handshake failed or the listener closed, for example, the error interface would be non-`nil`.

The connection interface's underlying type is a pointer to a `net.TCPConn` object because you're accepting TCP connections. The connection interface represents the server's side of the TCP connection. In most cases, `net.Conn` provides all methods you'll need for general interactions with the client. However, the `net.TCPConn` object provides additional functionality we'll cover in Chapter 4 should you require more control.

To concurrently handle client connections, you spin off a goroutine to asynchronously handle each connection ❹ so your listener can ready itself for the next incoming connection. Then you call the connection's `Close` method ❺ before the goroutine exits to gracefully terminate the connections by sending a FIN packet to the server.

Establishing a Connection with a Server

From the client's side, Go's standard library `net` package makes reaching out and establishing a connection with a server a simple matter. Listing 3-3 is a test that demonstrates the process of initiating a TCP connection with a server listening to 127.0.0.1 on a random port.

```
package ch03

import (
    "io"
    "net"
    "testing"
)

func TestDial(t *testing.T) {
    // Create a listener on a random port.
    listener, err := net.Listen("tcp", "127.0.0.1:")
    if err != nil {
        t.Fatal(err)
    }

    done := make(chan struct{})
❶   go func() {
        defer func() { done <- struct{}{} }()

        for {
            conn, err := ❷listener.Accept()
            if err != nil {
                t.Log(err)
                return
            }
```

```
        ❸  go func(c net.Conn) {
               defer func() {
                   c.Close()
                   done <- struct{}{}
               }()

               buf := make([]byte, 1024)
               for {
                   n, err := ❹c.Read(buf)
                   if err != nil {
                       if err != io.EOF {
                           t.Error(err)
                       }
                       return
                   }

                   t.Logf("received: %q", buf[:n])
               }
           }(conn)
       }
   }()

  ❺conn, err := net.Dial("❻tcp", ❼listener.Addr().String())
   if err != nil {
       t.Fatal(err)
   }

❽  conn.Close()
   <-done
❾  listener.Close()
   <-done
}
```

*Listing 3-3: Establishing a connection to 127.0.0.1 (*dial_test.go*)*

You start by creating a listener on the IP address 127.0.0.1, which the client will connect to. You omit the port number altogether, so Go will randomly pick an available port for you. Then, you spin off the listener in a goroutine ❶ so you can work with the client's side of the connection later in the test. The listener's goroutine contains code like Listing 3-2's for accepting incoming TCP connections in a loop, spinning off each connection into its own goroutine. (We often call this goroutine a *handler*. I'll explain the implementation details of the handler shortly, but it will read up to 1024 bytes from the socket at a time and log what it received.)

The standard library's `net.Dial` function is like the `net.Listen` function in that it accepts a network ❻ like `tcp` and an IP address and port combination ❼—in this case, the IP address and port of the listener to which it's trying to connect. You can use a hostname in place of an IP address and a service name, like *http*, in place of a port number. If a hostname resolves to more than one IP address, Go will attempt a connection to each one in order until a connection succeeds or all IP addresses have been exhausted. Since IPv6 addresses include colon delimiters, you must enclose an IPv6 address

in square brackets. For example, `"[2001:ed27::1]:https"` specifies port 443 at the IPv6 address 2001:ed27::1. `Dial` returns a connection object ❺ and an `error` interface value.

Now that you've established a successful connection to the listener, you initiate a graceful termination of the connection from the client's side ❽. After receiving the FIN packet, the `Read` method ❹ returns the `io.EOF` error, indicating to the listener's code that you closed your side of the connection. The connection's handler ❸ exits, calling the connection's `Close` method on the way out. This sends a FIN packet to your connection, completing the graceful termination of the TCP session.

Finally, you close the listener ❾. The listener's `Accept` method ❷ immediately unblocks and returns an error. This error isn't necessarily a failure, so you simply log it and move on. It doesn't cause your test to fail. The listener's goroutine ❶ exits, and the test completes.

Understanding Time-outs and Temporary Errors

In a perfect world, your connection attempts will immediately succeed, and all read and write attempts will never fail. But you need to hope for the best and prepare for the worst. You need a way to determine whether an error is temporary or something that warrants termination of the connection altogether. The `error` interface doesn't provide enough information to make that determination. Thankfully, Go's `net` package provides more insight if you know how to use it.

Errors returned from functions and methods in the `net` package typically implement the `net.Error` interface, which includes two notable methods: `Timeout` and `Temporary`. The `Timeout` method returns `true` on Unix-based operating systems and Windows if the operating system tells Go that the resource is temporarily unavailable, the call would block, or the connection timed out. We'll touch on time-outs and how you can use them to your advantage a bit later in this chapter. The `Temporary` method returns `true` if the error's `Timeout` function returns `true`, the function call was interrupted, or there are too many open files on the system, usually because you've exceeded the operating system's resource limit.

Since the functions and methods in the `net` package return the more general `error` interface, you'll see the code in this chapter use type assertions to verify you received a `net.Error`, as in Listing 3-4.

```
if nErr, ok := err.(net.Error); ok && !nErr.Temporary() { return err }
```

Listing 3-4: Asserting a `net.Error` to check whether the error was temporary

Robust network code won't rely exclusively on the `error` interface. Rather, it will readily use `net.Error`'s methods, or even dive in further and assert the underlying `net.OpError` struct, which contains more details about the connection, such as the operation that caused the error, the network type, the source address, and more. I encourage you to read the `net.OpError` documentation (available at *https://golang.org/pkg/net/#OpError/*) to learn more about specific errors beyond what the `net.Error` interface provides.

Timing Out a Connection Attempt with the DialTimeout Function

Using the `Dial` function has one potential problem: you are at the mercy of the operating system to time out each connection attempt. For example, if you use the `Dial` function in an interactive application and your operating system times out connection attempts after two hours, your application's user may not want to wait that long, much less give your app a five-star rating.

To keep your applications predictable and your users happy, it'd be better to control time-outs yourself. For example, you may want to initiate a connection to a low-latency service that responds quickly if it's available. If the service isn't responding, you'll want to time out quickly and move onto the next service.

One solution is to explicitly define a per-connection time-out duration and use the `DialTimeout` function instead. Listing 3-5 implements this solution.

```
package ch03

import (
    "net"
    "syscall"
    "testing"
    "time"
)

func ❶DialTimeout(network, address string, timeout time.Duration,
) (net.Conn, error) {
    d := net.Dialer{
    ❷  Control: func(_, addr string, _ syscall.RawConn) error {
            return &net.DNSError{
                Err:         "connection timed out",
                Name:        addr,
                Server:      "127.0.0.1",
                IsTimeout:   true,
                IsTemporary: true,
            }
        },
        Timeout: timeout,
    }
    return d.Dial(network, address)
}

func TestDialTimeout(t *testing.T) {
    c, err := DialTimeout("tcp", "10.0.0.1:http", ❸5*time.Second)
    if err == nil {
        c.Close()
        t.Fatal("connection did not time out")
    }
    nErr, ok := ❹err.(net.Error)
    if !ok {
        t.Fatal(err)
    }
```

```
    if ❺!nErr.Timeout() {
        t.Fatal("error is not a timeout")
    }
}
```

*Listing 3-5: Specifying a time-out duration when initiating a TCP connection (*dial_timeout_test.go*)*

Since the `net.DialTimeout` function ❶ does not give you control of its `net.Dialer` to mock the dialer's output, you're using our own implementation that matches the signature. Your `DialTimeout` function overrides the `Control` function ❷ of the `net.Dialer` to return an error. You're mocking a DNS time-out error.

Unlike the `net.Dial` function, the `DialTimeout` function includes an additional argument, the time-out duration ❸. Since the time-out duration is five seconds in this case, the connection attempt will time out if a connection isn't successful within five seconds. In this test, you dial 10.0.0.0, which is a non-routable IP address, meaning your connection attempt assuredly times out. For the test to pass, you need to first use a type assertion to verify you've received a `net.Error` ❹ before you can check its `Timeout` method ❺.

If you dial a host that resolves to multiple IP addresses, Go starts a connection race between each IP address, giving the primary IP address a head start. The first connection to succeed persists, and the remaining contenders cancel their connection attempts. If all connections fail or time out, `net.DialTimeout` returns an error.

Using a Context with a Deadline to Time Out a Connection

A more contemporary solution to timing out a connection attempt is to use a context from the standard library's `context` package. A *context* is an object that you can use to send cancellation signals to your asynchronous processes. It also allows you to send a cancellation signal after it reaches a deadline or after its timer expires.

All cancellable contexts have a corresponding `cancel` function returned upon instantiation. The `cancel` function offers increased flexibility since you can optionally cancel the context before the context reaches its deadline. You could also pass along its `cancel` function to hand off cancellation control to other bits of your code. For example, you could monitor for specific signals from your operating system, such as the one sent to your application when a user presses the CTRL-C key combination, to gracefully abort connection attempts and tear down existing connections before terminating your application.

Listing 3-6 illustrates a test that accomplishes the same functionality as `DialTimeout`, using `context` instead.

```
package ch03

import (
    "context"
    "net"
    "syscall"
```

```
    "testing"
    "time"
)

func TestDialContext(t *testing.T) {
 ❶ dl := time.Now().Add(5 * time.Second)
 ❷ ctx, cancel := context.WithDeadline(context.Background(), dl)
 ❸ defer cancel()

    var d net.Dialer // DialContext is a method on a Dialer
    d.Control = ❹func(_, _ string, _ syscall.RawConn) error {
        // Sleep long enough to reach the context's deadline.
        time.Sleep(5*time.Second + time.Millisecond)
        return nil
    }
    conn, err := d.DialContext(❺ctx, "tcp", "10.0.0.0:80")
    if err == nil {
        conn.Close()
        t.Fatal("connection did not time out")
    }
    nErr, ok := err.(net.Error)
    if !ok {
        t.Error(err)
    } else {
        if !nErr.Timeout() {
            t.Errorf("error is not a timeout: %v", err)
        }
    }
❻   if ctx.Err() != context.DeadlineExceeded {
        t.Errorf("expected deadline exceeded; actual: %v", ctx.Err())
    }
}
```

*Listing 3-6: Using a context with a deadline to time out the connection attempt (*dial_context_test.go*)*

Before you make a connection attempt, you create the context with a deadline of five seconds into the future ❶, after which the context will automatically cancel. Next, you create the context and its `cancel` function by using the `context.WithDeadline` function ❷, setting the deadline in the process. It's good practice to defer the `cancel` function ❸ to make sure the context is garbage collected as soon as possible. Then, you override the dialer's `Control` function ❹ to delay the connection just long enough to make sure you exceed the context's deadline. Finally, you pass in the context as the first argument to the `DialContext` function ❺. The sanity check ❻ at the end of the test makes sure that reaching the deadline canceled the context, not an erroneous call to `cancel`.

As with `DialTimeout`, if a host resolves to multiple IP addresses, Go starts a connection race between each IP address, giving the primary IP address a head start. The first connection to succeed persists, and the remaining contenders cancel their connection attempts. If all connections fail or the context reaches its deadline, `net.Dialer.DialContext` returns an error.